AUTOMATED THREAT MODELING FOR LLM-ENABLED APPLICATIONS USING LOCAL LARGE LANGUAGE MODELS AND DREAD RISK ASSESSMENT

by
BURAK İZER

Submitted to the Graduate School of Engineering and Natural Sciences
in partial fulfilment of
the requirements for the degree of Master of Science
in Cyber Security

Sabancı University
July 2026

AUTOMATED THREAT MODELING FOR LLM-ENABLED APPLICATIONS USING LOCAL LARGE LANGUAGE MODELS AND DREAD RISK ASSESSMENT

Approved by:

Prof. SÜHA ORHUN MUTLUERGIL .....
(Thesis Supervisor)

Assoc. Prof. FEYZULLAH ORÇUN ÇETIN .....

Asst. Prof. JULIO HERNANDEZ-CASTRO .....

Date of Approval: July 17, 2026

ABSTRACT

AUTOMATED THREAT MODELING FOR LLM-ENABLED APPLICATIONS USING LOCAL LARGE LANGUAGE MODELS AND DREAD RISK ASSESSMENT

BURAK İZER

Cyber Security, M.Sc. Thesis, July 2026

Thesis Supervisor: Prof. SÜHA ORHUN MUTLUERGIL

Keywords: Large Language Models, Threat Modeling, OWASP Web/API/LLM Top 10 Vulnerabilities, DREAD Risk Assessment, LoRA-based Fine-Tuning

Threat modeling is valuable during secure design, yet many approaches remain difficult to apply because they require manual DFD construction, security-taxonomy expertise, and interpretation of generic outputs. This limitation is sharper for LLM-integrated systems, where web, API, and LLM components create overlapping attack surfaces often analyzed through separate workflows. It is intensified by rapidly evolving LLM capabilities, integrations, and attack techniques, which continually expand the threat landscape.

This thesis investigates to what extent local LLMs can improve threat modeling for LLM-integrated systems within a questionnaire-driven pipeline that grounds and validates their output. A single adaptive questionnaire provides the application context for automatic static DFD generation, unified OWASP Web/API/LLM candidate-risk mapping, and reproducible DREAD-based risk scoring. Local LLMs are then used for threat identification and mitigation generation, adding system-specific threat descriptions, abuse paths, control gaps, and actionable mitigations to the questionnaire-derived model. Their outputs are validated against the generated DFD and candidate risk set to prevent unsupported risk codes, invalid diagram references, and LLM-driven changes to DREAD severity.

For sustainability, the thesis also considers an update pipeline in which trusted security sources are monitored for emerging LLM attack patterns, mitigation knowledge is converted into training data, and the local LLM is periodically updated with new threat types. Evaluation uses automatic structural metrics to compare the deterministic fallback, a base local LLM, and an optionally fine-tuned local LLM under the same inputs, assessing DFD integrity, risk coverage, grounding validity, schema conformance, system-specificity, mitigation actionability, and severity stability.

ÖZET

LLM-DESTEKLI UYGULAMALAR IÇIN YEREL BÜYÜK DIL MODELLERI VE DREAD RISK DEĞERLENDIRMESI KULLANARAK OTOMATIK TEHDIT MODELLEME

BURAK İZER

Siber Güvenlik, Yüksek Lisans Tezi, Temmuz 2026

Tez Danışmanı: Prof. Dr. SÜHA ORHUN MUTLUERGIL

Anahtar Kelimeler: Büyük Dil Modelleri, Tehdit Modelleme, OWASP Web/API/LLM İlk 10 Zafiyetleri, DREAD Risk Değerlendirmesi, LoRA Tabanlı İnce Ayar

Tehdit modelleme, güvenli tasarım sürecinde önemli bir değer sağlar; ancak mevcut birçok yaklaşım, manuel DFD oluşturma, güvenlik taksonomisi uzmanlığı ve genel nitelikli çıktıların yorumlanmasını gerektirdiği için pratikte uygulanması zor kalmaktadır. Bu sınırlılık, web, API ve LLM bileşenlerinin çoğu zaman ayrı iş akışlarıyla analiz edilen örtüşen saldırı yüzeyleri oluşturduğu LLM-entegre sistemlerde daha belirgin hale gelmektedir. Bu durum, LLM yeteneklerinin, entegrasyonlarının ve saldırı tekniklerinin hızlı biçimde gelişmesiyle daha da güçleşmekte ve tehdit ortamını sürekli genişletmektedir.

Bu tez, yerel LLM'lerin LLM-entegre sistemlere yönelik tehdit modelleme sürecini ne ölçüde iyileştirebileceğini, çıktıları temellendiren ve doğrulayan anket güdümlü bir pipeline içerisinde incelemektedir. Tek bir adaptif 91 soruluk anket, uygulama bağlamını sağlayarak otomatik statik DFD üretimini, birleşik OWASP Web/API/LLM aday risk eşlemesini ve yeniden üretilebilir DREAD tabanlı risk skorlamasını yönlendirmektedir. Yerel LLM'ler daha sonra tehdit tanımlama ve mitigasyon üretimi için kullanılmakta; anketten türetilen modele sisteme özgü tehdit açıklamaları, kötüye kullanım yolları, kontrol eksiklikleri ve uygulanabilir mitigasy-

onlar eklemektedir. Bu çıktılar, desteklenmeyen risk kodlarını, geçersiz diyagram referanslarını ve DREAD şiddet seviyesinde LLM kaynaklı değişiklikleri önlemek için üretilen DFD ve aday risk kümesiyle doğrulanmaktadır.

Sürdürülebilirliği ele almak için tez ayrıca, güvenilir güvenlik kaynaklarının yeni ortaya çıkan LLM saldırı desenleri açısından izlendiği, mitigasyon bilgisinin eğitim verisine dönüştürüldüğü ve yerel LLM'nin yeni tehdit türleriyle periyodik olarak güncellendiği bir güncelleme pipeline'ını da değerlendirmektedir. Değerlendirme, deterministik fallback'i, temel yerel LLM'yi ve opsiyonel olarak fine-tune edilmiş yerel LLM'yi aynı girdiler altında karşılaştırmak için otomatik yapısal metrikler kullanmakta; DFD bütünlüğü, risk kapsamı, temellendirme geçerliliği, şema uyumu, sisteme özgülük, mitigasyon uygulanabilirliği ve şiddet seviyesi kararlılığını ölçmektedir.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my thesis advisor, Prof. Süha Orhun Mutluergil, for his guidance, feedback, and support throughout this study. I am also grateful to my thesis committee members, Assoc. Prof. Feyzullah Orçun Çetin and Asst. Prof. Julio Hernandez-Castro, for their valuable time and constructive comments.

I would like to thank my family for their continuous encouragement, patience, and support during my graduate studies. Their belief in me has been a constant source of motivation.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

API Application Programming Interface.

CVE Common Vulnerabilities and Exposures.

CVSS Common Vulnerability Scoring System.

DFD Data Flow Diagram.

DREAD Damage, Reproducibility, Exploitability, Affected Users, and Discoverability.

LINDDUN Linkability, Identifiability, Non-repudiation, Detectability, Disclosure of information, Unawareness, and Non-compliance.

LLM Large Language Model.

LoRA Low-Rank Adaptation.

ML Machine Learning.

OWASP Open Worldwide Application Security Project.

QLoRA Quantized Low-Rank Adaptation.

RAG Retrieval-Augmented Generation.

RQ Research Question.

SQL Structured Query Language.

STRIDE Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.

1. INTRODUCTION

Large language models (LLMs) are increasingly embedded in software systems that do more than generate text. Modern LLM-enabled applications may receive input through web interfaces, expose API endpoints, retrieve context from internal knowledge bases, call external services, trigger workflow actions, and retain conversation or telemetry data for later use. As a result, their security posture cannot be understood by examining only the model layer. Recent work argues that LLM-based systems should be assessed at the system level because their risks combine conventional software-platform threats, adversarial machine-learning concerns, and prompt- or conversation-driven attack paths Nagaraja and Bahsi, 2026. In practice, these applications combine traditional web risks, API security weaknesses, and LLM-specific failure modes such as prompt injection, unsafe tool use, sensitive information exposure, retrieval contamination, and insecure handling of model outputs "Securing Large Language Models: Threats, Vulnerabilities and Responsible Practices", 2024; Tete, 2024.

This attack surface is also not static. LLM capabilities, application integration patterns, tool-use mechanisms, retrieval architectures, and adversarial techniques continue to evolve rapidly. Prompt injection research shows how user-supplied or retrieved instructions can manipulate model behavior in ways that differ from conventional input-validation failures "Formalizing and Benchmarking Prompt Injection Attacks and Defenses", 2023. Similarly, retrieval-augmented generation introduces knowledge-corruption and retrieval-poisoning risks in which compromised external or indexed content can influence downstream model outputs "PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation", 2024. New forms of indirect instruction manipulation, agent/tool abuse, retrieval poisoning, and data-exposure scenarios can therefore emerge after a threat-modeling workflow has already been designed. Therefore, an LLM-assisted threat-modeling approach should not only produce grounded results for a given system snapshot, but should also provide a path for keeping its threat knowledge current as the threat landscape changes

"Unique Security and Privacy Threats of Large Language Models: A Comprehensive Survey", 2025.

Threat modeling provides value during secure design by helping practitioners reason about assets, trust boundaries, data flows, entry points, controls, and likely abuse paths Hajrić et al., 2020; Naik et al., 2024. Many established approaches rely on architecture representations such as data-flow diagrams (DFDs) and threat taxonomies such as STRIDE, while risk-assessment methods such as DREAD are often used to support prioritization Hajrić et al., 2020; Naik et al., 2024. However, applying threat modeling in practice remains difficult. Conventional workflows often require manual DFD construction, familiarity with threat taxonomies, and analyst judgment to translate architecture features into meaningful findings. Prior work on threat-modeling tools reports challenges around tool usability, cognitive load, diagram-centered workflows, and context-agnostic or boilerplate threat outputs Bygdås et al., 2021; Shi et al., 2022; Thompson et al., 2024. These barriers are amplified by time cost, expert dependency, and the difficulty of maintaining threat modeling activities within agile development workflows Bernsmed et al., n.d.; "Threat Modeling", 2018.

This challenge becomes more pronounced for LLM-enabled applications. Their attack surface is hybrid by nature. A single user prompt may cross a web boundary, reach an API layer, enter an orchestration component, retrieve untrusted or sensitive context, invoke a model provider, and produce output that is consumed by downstream tools or business logic. Existing security taxonomies are useful, but they are often applied separately: web risks through web application categories, API risks through API-specific categories, and LLM risks through LLM-specific guidance. Recent LLM threat-modeling studies show the value of mapping LLM-specific risks to concrete DFD components and data flows, but they also illustrate that such analysis is often manual, domain-specific, or dependent on prepared architecture artifacts Nagaraja and Bahsi, 2025; Tete, 2024. This separation makes it harder to produce a single grounded view of how risks manifest across the whole system.

The motivation for this thesis comes from this gap. The aim is not to replace established threat-modeling tools or to perform a feature-by-feature comparison against them. Rather, this work investigates to what extent local Large Language Models can improve threat modeling for LLM-integrated systems when their outputs are grounded and validated within a questionnaire-driven pipeline. Existing DFD- and STRIDE-centered tools illustrate why manual diagramming and expertise-heavy workflows can be challenging, but they are used as motivation rather than as experimental baselines in this thesis Bygdås et al., 2021; Shi et al., 2022; Thompson

et al., 2024.

In this thesis, generic LLM-enabled applications refer to systems that are not restricted to a single vertical domain, such as healthcare or finance, but share common architectural features such as web or API entry points, LLM orchestration, retrieval or memory components, tool use, external services, and logging or monitoring pipelines.

1.1 Problem Statement

LLM-enabled applications require threat modeling that can reason across web, API, and LLM-specific security surfaces at the same time. However, four practical problems limit the usefulness of many existing approaches for this setting.

First, threat modeling often begins with manual architecture modeling. If the data-flow diagram is incomplete or inconsistent, subsequent threat analysis inherits those weaknesses. This is especially problematic for LLM-enabled applications, where important components such as model gateways, retrieval pipelines, vector stores, tool execution layers, logging pipelines, and external model providers may be omitted or modeled inconsistently. Prior taxonomies of threat-modeling tools emphasize that modeling style and tool support strongly shape the quality and completeness of the resulting analysis Shi et al., 2022.

Second, risk identification and risk prioritization are often weakly connected to the concrete facts that describe a system. A generic threat may be technically relevant but still lack evidence, an affected component, a plausible abuse path, or a clear control gap. Without those links, the output is difficult to audit and difficult to act on. This is a known limitation of overly generic threat lists and boilerplate tool output, which can overwhelm users without providing enough system-specific context Thompson et al., 2024.

Third, LLMs can help generate richer explanations and mitigations, but free-form LLM use introduces reliability problems. Existing LLM-assisted threat-modeling work demonstrates the potential of prompt engineering and fine-tuning for threat identification and mitigation mapping, while also highlighting the need for structured outputs and domain constraints Mollaeefar et al., 2024; Wu et al., 2024. In

security work, an unconstrained model may invent risks, refer to non-existent architecture components, assign unsupported severity levels, or produce generic advice that is not grounded in the target system. Such hallucinations are not merely cosmetic; they can mislead prioritization and obscure the actual design risks. Work on structured LLM generation for formal threat-modeling artifacts similarly motivates validation mechanisms that check consistency and constrain unsupported model outputs Pathe, 2025.

Fourth, the knowledge needed for LLM application security changes over time. Static prompts, fixed examples, and outdated training data may fail to capture newly observed attack vectors or mitigation patterns. A threat-modeling tool for LLM-integrated systems therefore needs a sustainable update mechanism that can incorporate trusted security knowledge about emerging attacks without sacrificing grounding, traceability, or reproducibility. Parameter-efficient adaptation methods such as LoRA make task-specific model updates more practical than full model retraining, and efficient fine-tuning techniques further support the feasibility of periodically adapting local models Dettmers et al., 2023; Hu et al., 2022.

1.2 Proposed Approach

This thesis proposes and evaluates an automated threat-modeling workflow for generic LLM-enabled applications. The system is implemented as a Flask-based web application, but the main contribution is the pipeline design rather than the web interface itself. The workflow is designed to study how local LLMs can improve threat identification and mitigation generation while keeping architecture modeling, candidate-risk mapping, and severity scoring reproducible.

An adaptive questionnaire of up to 91 questions captures architecture, data handling, model usage, API exposure, retrieval behavior, tool use, controls, deployment characteristics, and DREAD-relevant risk signals. A deterministic static mapper then derives a data-flow diagram from the questionnaire answers. The LLM is not involved in data-flow diagram construction.

Next, a deterministic risk catalog maps answer-derived evidence to candidate risks across OWASP LLM, OWASP Web, and OWASP API categories. A local LLM is used for threat identification within this candidate set. It adds system-specific threat

descriptions, affected data-flow diagram nodes or edges, evidence, abuse paths, control gaps, and confidence values for the questionnaire-derived risks.

A grounding validator then removes hallucinated risk codes, strips references to non-existent data-flow diagram elements, downgrades unsupported findings, and records candidates that the model failed to address. After validation, a deterministic DREAD scoring layer computes risk severity from questionnaire-derived signals. The LLM never computes or changes severity. DREAD has previously been used in LLM-related risk assessment, but this thesis uses it as a questionnaire-grounded deterministic scoring layer rather than as a manual expert-scoring exercise Tete, 2024; Zahid et al., 2024. Finally, a local LLM generates mitigation actions for validated and scored threats. Each mitigation must reference the supplied risk code, affected component, evidence, or control gap.

The local LLM is therefore used for the language-rich parts of threat modeling: explaining how a candidate risk manifests in a particular system and producing concrete mitigation guidance. If the LLM is unavailable or produces unusable output, the system falls back to a deterministic baseline and still produces a valid risk artifact.

To address sustainability, this thesis also considers an update pipeline for keeping the local LLM informed about new LLM attack patterns. In this pipeline, trusted security sources are monitored for emerging attack vectors and mitigation guidance. Relevant attack descriptions, affected components, exploitation patterns, control gaps, and mitigations are normalized into task-specific training examples for the threat identification and mitigation generation stages. This allows the local LLM to be periodically updated with new threat types while the generated outputs remain validated against the questionnaire-derived DFD, the candidate risk set, and the deterministic DREAD scores.

1.3 Research Questions

The main research question of this thesis is:

> To what extent can local Large Language Models improve the threat modeling process for LLM-integrated systems?

This question is decomposed into four research questions (RQs):

RQ1: To what extent can local Large Language Models identify threats against LLM-integrated systems during the threat modeling process?

RQ2: How can the effectiveness of local Large Language Models in identifying threats against LLM-integrated systems be improved?

RQ3: To what extent can local Large Language Models generate relevant mitigation strategies for threats identified in LLM-integrated systems?

RQ4: How can an LLM-assisted threat modeling tool for LLM-integrated systems keep up to date with emerging attack vectors and evolving threat patterns?

1.4 Contributions

This thesis makes the following contributions.

First, it presents a unified workflow for modeling the combined Web, API, and LLM attack surface of generic LLM-enabled applications in a single grounded pipeline.

Second, it introduces an adaptive questionnaire-driven approach that connects system description, architecture modeling, and candidate-risk mapping without requiring the user to construct a data-flow diagram manually.

Third, it implements a questionnaire-based instantiation of the DREAD risk assessment, where severity is computed deterministically from response-derived signals rather than assigned by the LLM.

Fourth, it uses local LLMs for threat identification and mitigation generation, allowing the model to add system-specific threat descriptions, abuse paths, control gaps, and mitigation details to questionnaire-derived risks.

Fifth, it provides a grounding validator that acts as a guardrail against hallucinated risk codes, invalid architecture references, unsupported findings, and LLM-driven severity changes.

Sixth, it defines a sustainability-oriented update pipeline in which trusted security

sources can be used to collect emerging LLM attack and mitigation knowledge and convert it into task-specific training data for periodically updating the local LLM.

Finally, it prepares supervised fine-tuning datasets for the LLM-supported threat identification and mitigation generation stages, and evaluates fine-tuning through structural and grounding-oriented metrics.

1.5 Scope and Evaluation

The evaluation in this thesis is intentionally structural and automatic. It does not rely on human raters, expert-labeled gold standards, or external-tool comparison. Instead, the system is evaluated internally by comparing the deterministic fallback, a base local LLM, and a fine-tuned local LLM under the same inputs. The evaluation focuses on data-flow diagram integrity, candidate-risk coverage, grounding validity, schema conformance, system-specificity, mitigation actionability, and the prevention of LLM-driven severity changes.

System-specificity and mitigation actionability are not evaluated as expert-judged semantic quality. Instead, they are assessed through structural proxy metrics, such as references to concrete data-flow diagram nodes, answer-derived evidence, target components, validation steps, and evidence-linked mitigation fields.

The sustainability component is treated as a design and data-preparation objective rather than as a long-term autonomous monitoring deployment. The thesis considers how trusted security knowledge about emerging LLM attacks can be transformed into training data for the LLM-supported stages, while preserving the same grounding and validation constraints used in the main pipeline.

This scope is deliberate. Because the deterministic pipeline is the source of data-flow diagram construction, candidate-risk mapping, and DREAD scoring, reproducibility claims apply to those deterministic stages. The LLM-supported stages are treated as enrichment stages and are evaluated for conformance, grounding, and specificity rather than for expert-judged semantic superiority. Fine-tuning is therefore investigated as an experimental extension to improve the structure and groundedness of the local LLM outputs, not as a replacement for deterministic risk analysis.

2. BACKGROUND

This chapter provides the technical background for the thesis. It first introduces threat modeling as a secure design activity and explains the role of system decomposition, Data Flow Diagrams (DFDs), trust boundaries, threat elicitation, and risk assessment. Then it discusses the security characteristics of LLM-integrated systems, whose attack surface combines conventional software and API risks with LLM-specific threats such as prompt injection, insecure output handling, retrieval poisoning, sensitive information disclosure, and tool misuse.

The chapter also introduces DREAD as the risk assessment model used in this thesis and reviews how previous work has applied structured risk assessment to LLM-related threats. After establishing these foundations, the chapter discusses automated and LLM-assisted threat modeling, including approaches that use LLMs for DFD generation, threat classification, and mitigation mapping. Finally, it reviews local and fine-tuned LLMs as a basis to improve threat identification and mitigation generation, before concluding with related work and the research gap addressed by this thesis.

The purpose of this chapter is not only to define the main concepts used later in the methodology, but also to position the proposed approach within existing research. In particular, it highlights the need for a threat modeling workflow that can address LLM-integrated systems at the system level while preserving reproducibility, grounding, and risk-scoring consistency.

2.1 Threat Modeling Foundations

Threat modeling is a secure design activity that is used to identify, analyze, and prioritize potential threats before security problems become implementation-level failures. Instead of treating security as a final testing step, threat modeling places security reasoning within the design process by identifying assets, system components, trust boundaries, data flows, and possible attacker behaviors (Bygdås et al., 2021; Hajrić et al., 2020).

A typical threat modeling process begins with system decomposition. The system is commonly represented through actors, processes, data stores, data flows, and trust boundaries, often using Data Flow Diagrams. Once this system model is established, analysts can elicit threats using a threat taxonomy or methodology, and then apply a risk assessment model to prioritize the identified threats. In this thesis, DREAD is the relevant risk assessment model because it provides explicit dimensions for reasoning about Damage, Reproducibility, Exploitability, Affected Users, and Discoverability (Hajrić et al., 2020; Shi et al., 2022).

Traditional tools such as Microsoft Threat Modeling Tool and OWASP Threat Dragon demonstrate that DFD-based modeling and structured threat identification are established practices in secure software design. These tools help analysts represent system components and reason about potential threats, but they still depend on the user to provide an accurate model of the system (Bygdås et al., 2021; Shi et al., 2022).

Prior work identifies several limitations in traditional and tool-supported threat modeling. Manual DFD construction can be time-consuming and difficult, especially when teams lack security expertise or when the system contains many components and communication paths. Empirical studies report that teams may struggle with abstraction, spend significant time constructing DFDs, and experience threat modeling as a burden alongside development work (Bernsmed et al., n.d.; "Threat Modeling", 2018).

Tool support does not fully remove these problems. Existing tools may produce context-agnostic findings, false positives, or long lists of suggested threats and mitigations that require further interpretation before they become useful (Bernsmed et al., n.d.; Thompson et al., 2024). These limitations motivate automation, but they also show why automation must preserve architectural traceability and avoid

producing findings detached from the system being modeled.

2.2 Security of LLM-Integrated Systems

LLM-integrated systems are not only language models exposed through an interface. They are software systems in which an LLM is embedded into a broader application architecture that may include web front ends, API gateways, prompt construction logic, retrieval components, vector stores, memory, external tools, third-party services, logging pipelines, and user-facing output channels. As a result, their security cannot be evaluated only at the model level. The surrounding application structure determines what data reaches the model, what actions the model can influence, and how model outputs affect users or downstream systems.

Several system-level studies emphasize that LLM-based systems combine threat landscapes that are often analyzed separately. Conventional software and Web/API vulnerabilities may interact with adversarial machine learning threats and conversational attacks such as prompt injection or jailbreaks. Security analysis must therefore consider how conventional platform attacks, adversarial ML attacks, and prompt-based attacks interact across the end-to-end system rather than treating model behavior in isolation (Nagaraja & Bahsi, 2025, 2026). This is especially important when LLM outputs are connected to external tools, business logic, retrieval pipelines, or sensitive data stores.

LLM-specific risks also create bridges back into traditional application security. Insecure output handling, prompt injection, sensitive information disclosure, excessive agency, retrieval poisoning, and tool misuse can lead to consequences that resemble conventional Web or API failures, including unauthorized data access, unsafe actions, or injection into downstream components. Work on LLM-powered applications discusses how LLM-specific risks can overlap with traditional vulnerabilities such as SQL injection, cross-site scripting, server-side request forgery, and other Web/API attack patterns (Tete, 2024). In addition, research on LLM-based healthcare systems maps OWASP LLM risks to concrete system components and data flows, showing that LLM threats must be analyzed in relation to the architecture around the model (Nagaraja & Bahsi, 2025).

Retrieval-augmented generation and tool-enabled architectures further widen this

attack surface. In RAG settings, the model may rely on external knowledge sources that can be manipulated, poisoned, or retrieved out of context. Prompt injection research also shows that malicious instructions can be embedded in inputs or retrieved content, causing the model to follow attacker-controlled instructions instead of developer or system intent ("Formalizing and Benchmarking Prompt Injection Attacks and Defenses", 2023; "PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation", 2024). These risks are not isolated model failures; they depend on how data flows through the surrounding application and how much authority the LLM-mediated workflow has over downstream actions.

For this reason, threat modeling for LLM-integrated systems must be system-level rather than model-only. It must account for user inputs, prompt construction, retrieval and memory components, tool calls, APIs, data stores, trust boundaries, and output handling. This motivates a workflow that can represent the application architecture explicitly and map LLM-specific risks onto concrete components and flows. In this thesis, this system-level view is necessary because the goal is to assess generic LLM-integrated systems whose security depends on the combined behavior of conventional software components and LLM-specific interactions.

2.3 Risk Assessment and DREAD

Identifying threats is not sufficient on its own; a threat modeling process also needs a way to prioritize them. Without risk assessment, the output of threat modeling can become a long list of possible issues with no clear indication of which risks should be addressed first. Risk assessment provides a structured way to compare threats according to their potential impact, ease of exploitation, repeatability, exposure, and discoverability.

DREAD is a risk assessment model that evaluates threats across five dimensions: Damage, Reproducibility, Exploitability, Affected Users, and Discoverability. Damage captures the potential harm caused by a successful attack. Reproducibility considers whether the attack can be repeated reliably. Exploitability reflects the effort or capability required to carry it out. Affected Users captures the possible blast radius, and Discoverability considers how easily an attacker can find or trigger the weakness. These dimensions make DREAD suitable for design-time reasoning because they separate impact, exposure, and practical attack feasibility (Hajrić et

al., 2020; Naik et al., 2024).

DREAD is especially useful in this thesis because the proposed system requires prioritization that is reproducible, auditable, and grounded in system evidence. The threat model is generated from questionnaire answers, and the risk score must therefore be explainable in terms of explicit system properties rather than analyst intuition alone. Unlike CVSS, which is primarily designed around known implementation vulnerabilities and CVE-like defects, DREAD can be adapted to reason about architectural and behavioral threats at design time. This is important for LLM-integrated systems, where risks may arise from prompt flow, retrieval behavior, tool authority, data exposure, user scale, and control maturity rather than from a single known implementation flaw.

Prior work has already applied DREAD to LLM-related threats. Zahid et al. use DREAD to assess a taxonomy of attacks against educational LLMs, showing that the model can be adapted to reason about LLM-specific risks such as prompt injection, adversarial prompts, token smuggling, and jailbreak-style attacks (Zahid et al., 2024). Tete also applies structured threat modeling and DREAD-based risk analysis to LLM-powered applications, further supporting the use of DREAD in this problem space (Tete, 2024).

However, existing uses of DREAD for LLM-related threats are often manual, qualitative, or expert-driven. In such approaches, risk scores depend on human interpretation of the threat and may vary across analysts or contexts. This is a limitation for an automated threat modeling workflow, because the same system description should not receive different severity scores simply because a different analyst or model phrased the threat differently. Therefore, this thesis uses DREAD as the scoring framework but implements it deterministically: the DREAD dimensions are derived from questionnaire-grounded system evidence, and the final risk level is computed from those dimensions rather than generated by the LLM.

This distinction is central to the proposed approach. The LLM may help identify grounded threats or generate mitigation text, but it does not assign final severity. DREAD scoring remains outside the LLM layer so that prioritization remains reproducible, auditable, and grounded across runs. This allows the system to benefit from LLM assistance while preserving deterministic risk assessment as the authoritative basis for severity.

2.4 Automated and LLM-Assisted Threat Modeling

The limitations of manual threat modeling have motivated research into automated and LLM-assisted approaches. Automation is attractive because threat modeling requires repeated interpretation of system structure, security-relevant assets, data flows, trust boundaries, and possible attacker behaviors. These tasks are difficult to perform consistently by hand, especially when systems are large, rapidly changing, or unfamiliar to the analyst. LLMs are therefore increasingly explored as assistants for converting system descriptions into structured security artifacts, classifying threats, and generating mitigation recommendations.

Recent work shows that LLMs can support different stages of the threat modeling workflow. PILLAR, for example, uses LLMs to automate parts of the LINDDUN privacy threat modeling process, including DFD generation, threat classification, and risk prioritization from natural language system descriptions (Mollaeefar et al., 2024). ThreatModeling-LLM applies LLMs to threat modeling in the banking domain and investigates prompt engineering and LoRA-based fine-tuning for improving threat identification and mitigation mapping (Wu et al., 2024). Other work explores the use of LLMs to create formal domain-specific threat modeling languages, showing that structured generation can reduce manual modeling effort while still requiring attention to hallucination control and consistency (Pathe, 2025).

These studies demonstrate that LLMs can reduce the burden of producing security models and threat descriptions. However, they also expose an important design choice: whether the LLM should generate the core system model or operate within a system model produced by another process. Approaches that generate DFDs or formal models directly from unstructured natural language can be flexible, but they may also create structural inconsistency, hallucinated components, or unsupported assumptions if the model output is not constrained and validated. In threat modeling, such errors are significant because downstream threats and mitigations depend on the correctness of the system representation.

For this reason, LLM-assisted threat modeling requires guardrails around both input structure and output validity. Structured prompting, schemas, and domain-specific output formats can reduce invalid responses, but they do not by themselves guarantee that a generated threat is grounded in the actual system. Prior work on LLM-based threat modeling notes challenges such as hallucination control, stylistic inconsistency, and subjective evaluation of generated outputs (Pathe, 2025). These

concerns are especially relevant when an LLM is used to identify threats or mitigations that may later influence risk prioritization.

The approach developed in this thesis follows a constrained form of LLM assistance. The LLM is not responsible for generating the DFD, deciding the candidate risk universe, or assigning final severity. Instead, automation is divided into deterministic and LLM-assisted stages: the questionnaire-derived system model and candidate risks provide the structure, while the LLM contributes grounded threat descriptions and mitigation text within that structure. This design aims to retain the flexibility of LLM-assisted analysis while preserving architectural traceability and preventing unsupported outputs from controlling the threat model.

2.5 Local LLMs, Fine-Tuning, and Sustainable Threat Adaptation

Local LLMs are relevant to this thesis because they allow threat modeling assistance to be explored without assuming dependence on a proprietary cloud model. A local or open-source model can be deployed within a controlled environment, configured consistently across experiments, and updated or replaced without changing the deterministic parts of the pipeline. This is important for a threat modeling workflow that must remain reproducible, auditable, and grounded even when the model layer changes.

At the same time, base LLMs may not be sufficiently reliable for specialized security tasks without adaptation. Threat identification and mitigation generation require the model to understand security terminology, system architecture, attack paths, control gaps, and structured output requirements. Prior work suggests that prompt engineering and fine-tuning can improve LLM performance in threat modeling tasks. ThreatModeling-LLM reports that combining prompt engineering with LoRA-based fine-tuning improved threat identification and mitigation mapping for a banking-system threat modeling task (Wu et al., 2024). More generally, LoRA provides a parameter-efficient fine-tuning method that adapts large language models by training low-rank update matrices rather than all model parameters (Hu et al., 2022). QLoRA further reduces the resource cost of fine-tuning by combining quantization with low-rank adaptation (Dettmers et al., 2023).

In this thesis, fine-tuning is treated as a mechanism for adapting a local model to

the specific tasks required by LLM-assisted threat modeling. For threat identification, it supports the goal of improving how effectively the model recognizes relevant threats in LLM-integrated systems. For mitigation generation, it enables comparison between base and adapted model behavior when producing relevant, actionable, and risk-linked mitigation strategies. The same mechanism also supports long-term maintainability: newly observed attack patterns and mitigation knowledge can be converted into task-specific training data and used to refresh the local model.

This update-oriented view is important because LLM-integrated systems face a changing threat landscape. New prompt injection techniques, retrieval attacks, tool-use abuse patterns, and deployment-specific weaknesses continue to appear. A sustainable workflow can treat trusted new threat knowledge as input to a controlled update process: collect relevant sources, normalize them into task-specific training examples, fine-tune or refresh the local model, and evaluate whether the updated model improves structural conformance, grounding, threat coverage, or mitigation relevance. This connects fine-tuning not only to model improvement, but also to the long-term maintenance of threat knowledge.

Fine-tuning does not remove the need for validation. A fine-tuned model can still produce unsupported claims, overfit to the training distribution, or generate plausible but invalid outputs. For that reason, the proposed workflow keeps the local LLM inside a validation boundary: the model may improve threat identification and mitigation generation, but the system model, candidate risk set, grounding checks, and DREAD scoring remain external to the model.

The value of the local LLM layer is therefore not that it replaces the threat modeling pipeline, but that it makes the pipeline adaptable. It allows the same deterministic workflow to benefit from model improvements, task-specific fine-tuning, model replacement through configuration, and periodic updates as the LLM threat landscape evolves. This keeps the LLM as the active improvement layer while preserving the reproducible, auditable, and grounded properties required for threat modeling.

2.6 Related Work and Research Gap

Recent research on LLM-integrated systems increasingly treats these applications as system-level security problems rather than isolated model-level artifacts. Nagaraja

and Bahsi (2026) argue that LLM-based systems combine conventional platform threats, adversarial machine learning threats, and conversational attacks, and that these risks should be assessed across the full system context rather than in isolation. Their healthcare-focused threat modeling study also shows that OWASP LLM risks can be mapped to concrete system components and data flows, supporting architecture-aware threat modeling for LLM applications (Nagaraja & Bahsi, 2025). This system-level perspective is central to the present thesis, because the relevant risks emerge from the interaction between the LLM, application logic, data flows, retrieval components, tools, and external services.

The same need for system-level reasoning has motivated work on automating parts of the threat modeling process. Mollaeefar et al. (2024) use LLMs to generate DFDs and privacy threat models from natural language system descriptions, demonstrating that LLMs can reduce the manual burden of system modeling and threat classification. Wu et al. (2024) apply LLMs to banking-system threat modeling and report that prompt engineering and LoRA-based fine-tuning improve threat identification and mitigation mapping. These studies show that LLMs can contribute meaningfully to threat modeling workflows, but they make different architectural choices from this thesis: PILLAR lets the LLM participate in generating the system model, while ThreatModeling-LLM relies on manually prepared DFD input and focuses on a banking-specific setting.

This thesis takes a different route. Instead of asking the LLM to generate the DFD or determine the risk universe, the proposed workflow derives the DFD and candidate risks deterministically from questionnaire answers. The local LLM is then used only after this structure exists, for grounded threat identification and mitigation generation. This preserves the benefit of LLM assistance while reducing the chance that hallucinated components, unsupported assumptions, or unstable severity judgments control the threat model.

Risk scoring provides another point of connection with the literature. Zahid et al. (2024) apply DREAD to a taxonomy of LLM attacks, demonstrating that DREAD can be used to reason about LLM-related threats. Tete (2024) also combines structured threat modeling with DREAD-style risk analysis for LLM-powered applications. Building on this direction, the present thesis uses DREAD not as a manual or qualitative judgment, but as a deterministic scoring mechanism grounded in questionnaire evidence. The LLM may describe the threat, but it does not assign the final severity.

Model deployment is also part of the research positioning. Hosted frontier models

offer strong general reasoning capability and are therefore a natural choice in many LLM-assisted security workflows. This thesis instead focuses on local or open-source LLMs, which creates a different trade-off around cost, privacy, reproducibility, and control. A smaller local model may be weaker than a large hosted model out of the box, but it can be run in a controlled environment, swapped through configuration, and adapted through task-specific fine-tuning. LoRA is especially relevant here because it enables efficient adaptation without retraining all model parameters (Hu et al., 2022). ThreatModeling-LLM provides supporting evidence that this type of adaptation can improve threat modeling outputs in a specialized domain (Wu et al., 2024).

The resulting research gap is therefore not simply that LLMs have not been used for threat modeling. Prior work already shows that they have. The gap is the lack of a generic, end-to-end, local-LLM-assisted workflow for LLM-integrated systems in which the core system model, candidate risk set, and DREAD severity calculation remain reproducible, auditable, and grounded. This thesis addresses that gap by combining deterministic questionnaire-driven DFD and risk-candidate generation with a constrained local LLM layer for threat identification and mitigation generation. Fine-tuning then provides a practical path for adapting the local model to the task and for updating the LLM-assisted layer as new LLM threat patterns emerge.

3. METHODOLOGY

3.1 System Overview and Design Goals

This thesis proposes a deterministic-first, questionnaire-driven threat-modeling system for generic LLM-enabled applications. The system is implemented as a Flask-based web application that collects structured information about a target system and transforms it into threat-modeling artifacts through a controlled pipeline. The central idea is to combine the repeatability of deterministic analysis with the contextual expressiveness of a local LLM, while preventing the LLM from becoming the authoritative source for architecture, risk selection, or severity assessment.

The methodological motivation for this design is the observation that LLM-enabled applications are rarely limited to a single model endpoint. In practice, they are usually composed of web interfaces, API endpoints, backend services, retrieval components, data stores, external integrations, logging mechanisms, authentication and authorization layers, and sometimes tool-calling or agentic execution capabilities. As a result, their attack surface is not only an LLM attack surface; it is a combined LLM, Web, and API attack surface. Existing approaches often address these areas through separate taxonomies or workflows, while domain-specific solutions do not always generalize to arbitrary LLM-enabled applications. This thesis therefore treats the target system as a complete application architecture rather than as an isolated prompt-response interaction.

The implemented system follows a pipeline that begins with an adaptive questionnaire and ends with a structured risk output. At a high level, the workflow consists of questionnaire collection, deterministic DFD generation, deterministic candidate risk mapping, constrained local LLM-based threat identification, grounding validation, deterministic DREAD-based scoring, and mitigation generation. Later sections de-

scribe these stages in detail. In this section, the emphasis is on the design principles that guide the overall methodology.

The first design goal is to reduce manual modeling effort. Traditional threat modeling often assumes that the analyst can manually construct a data-flow diagram, identify trust boundaries, select relevant threat categories, and estimate risk levels. These activities require security expertise and are difficult to perform consistently. The proposed system replaces manual diagram construction with a structured questionnaire. The user describes the application by answering concrete questions about its architecture, data flows, interfaces, controls, and operational context, and the system derives the downstream artifacts from those answers. This makes the workflow more accessible while still preserving a structured security methodology.

The second design goal is deterministic authority over reproducible decisions. Every artifact that must be regenerable from the same input is produced by the deterministic layer: the data-flow diagram is constructed from questionnaire answers by static mapping rules, the candidate risk space is selected through deterministic predicates over those answers, grounding is enforced by rule-based validation, and DREAD scores are computed from answer-derived signals. Because none of these steps depend on generative sampling, the same questionnaire response produces the same structural model, the same candidate risk space, and the same risk scores by construction. This property is what allows the core analysis to be evaluated with structural and automatic metrics rather than relying on the variable output of a language model.

The third design goal is unified treatment of LLM, Web, and API risk. The system does not divide the questionnaire into three separate independent surveys; instead, it encodes the combined surface by design. All 91 questionnaire items are LLM-scoped, while 62 also carry API scope and 51 also carry Web scope. This overlapping scope structure reflects the fact that LLM-enabled systems often expose risks through interactions between model behavior and the surrounding application infrastructure. For example, a prompt-injection issue may become more severe when combined with tool execution, weak API authorization, unsafe output rendering, or sensitive-data retrieval. The methodology therefore models the LLM layer together with the application layers that mediate access to it.

The fourth design goal is grounding and hallucination control. The local LLM is invoked only after the deterministic artifacts have already fixed the analysis space, so it never operates from an open-ended prompt. It receives a bounded set of candidate risks and a concrete DFD context, and every element it produces is checked against

that context before it can enter the final result. References to DFD nodes or edges that do not exist are stripped, findings that are not supported by the provided evidence are downgraded, and risk codes outside the deterministic candidate set are prevented from becoming authoritative primary risks. In this way the validation layer, rather than the model itself, decides which generated content is admissible.

The fifth design goal is a separation between qualitative explanation and quantitative prioritization. Threat identification and risk scoring are treated as related but distinct tasks. The LLM contributes the qualitative part: it describes how a threat could manifest in the specific system and why a missing control matters. The quantitative part – the final risk level – is derived exclusively from deterministic DREAD scoring over Damage, Reproducibility, Exploitability, Affected Users, and Discoverability, computed from questionnaire-derived signals. This is the single point at which the thesis draws its firmest line: the model may explain a threat, but it cannot make a risk Critical, High, Medium, or Low by assertion. Natural-language explanations remain valuable even where they are unsuitable as a direct numerical scoring mechanism.

The sixth design goal is model-agnostic local execution. The LLM layer is accessed through a configurable local interface, so the system can use different local models without changing the deterministic pipeline. This supports experimentation with base and fine-tuned models while keeping the core workflow stable. The system is also designed to remain usable when the LLM is unavailable: if the local model cannot be reached or the LLM threat-identification path fails, the pipeline falls back to deterministic risk analysis and still produces a valid output. In this design, the LLM is an enrichment layer rather than a dependency required for basic functionality.

The seventh design goal is traceability. The system preserves links between user answers and generated artifacts wherever possible. Questionnaire responses are stored in a canonical answer format, DFD nodes and edges are derived from answer-based architecture signals, candidate risks are connected to evidence questions, DREAD scoring is based on explicit answer-derived dimensions, and grounding validation records whether LLM-generated findings remain supported by the deterministic context. This traceability supports auditability: the output is not merely a generated report, but a structured result whose major elements can be traced back to the input.

These design goals map directly onto the research questions of this thesis. The deterministic-authority, unified-surface, and traceability goals (goals two, three, and seven) underpin RQ1, which asks to what extent a questionnaire-driven pipeline can

model the combined attack surface and produce reproducible scores without manual modeling. The separation of qualitative explanation from quantitative prioritization and the model-agnostic execution goals (goals five and six) correspond to RQ2, which concerns the additional value a constrained local LLM contributes to threat identification and mitigation. The grounding and hallucination-control goal (goal four) corresponds to RQ3, which examines whether the deterministic constraints prevent hallucinated risk codes, invalid DFD references, and LLM-driven changes to severity. Finally, the model-agnostic design also enables RQ4, the experimental question of whether fine-tuning a local model improves the structural conformance of its generated content relative to the base model.

Overall, the system is designed around a clear division of responsibilities. The deterministic components define the structure, scope, constraints, and scores of the threat model, while the local LLM contributes contextual reasoning and natural-language enrichment within those constraints. This division is the central methodological contribution of this thesis: it aims to make threat modeling for generic LLM-enabled applications more accessible and more repeatable, while still allowing the final report to contain system-specific threat descriptions and mitigation guidance.

3.2 Questionnaire Design

The questionnaire is the primary input mechanism and the methodological foundation of the proposed system. It defines what information is collected from the user, how that information is represented, and how later pipeline stages can use it. Instead of asking the user to manually draw a DFD or directly assign risk levels, the questionnaire asks concrete questions about the target application's purpose, exposed interfaces, data sources, model behavior, connected tools, API usage, output handling, security controls, and operational practices. These answers become the evidence base for the rest of the pipeline.

The implemented questionnaire contains 91 fixed questions. The question set is intentionally stable because the downstream deterministic components depend on predictable identifiers, options, categories, scopes, and scoring metadata. If questions could be added or changed dynamically during normal use, the DFD mapper, candidate risk catalog, and DREAD scoring logic would no longer operate over a stable input structure. For this reason, the system treats the questionnaire as a

fixed methodological instrument rather than as an editable runtime form.

The questionnaire contains 67 single-choice questions and 24 multi-choice questions. Single-choice questions are used when the system needs one clear answer about a property or control state, such as whether sensitive data is processed or whether a particular safeguard exists. Multi-choice questions are used when several conditions may apply at the same time, such as multiple exposed interfaces, multiple input sources, multiple external integrations, or multiple user groups. This distinction reflects the structure of real LLM-enabled applications, where several exposure paths and architectural patterns may coexist.

Each question is stored as a structured record. The main fields are the question identifier, question text, answer type, predefined options, category, scope, DREAD dimensions, and DREAD weights. This schema allows the questionnaire to serve several functions at once: the text and options support user interaction, the category and scope fields organize the question within the broader methodology, the DREAD metadata connects the answer to later scoring, and the fixed identifier allows deterministic services to read specific answers reliably.

The questionnaire uses 38 fine-grained categories. These categories are more detailed than broad labels such as "LLM security" or "API security"; they include areas such as system context, exposure and interfaces, data/RAG/memory, authentication and authorization, identity and tenant isolation, tool-calling and agency, API authorization, API inventory, web attack surface, output handling, output rendering, secrets and cryptography, logging and detection, resource consumption, supply chain, transport security, data lifecycle, and operational governance, among others. In the thesis narrative these fine-grained categories can be grouped into broader themes, but the implementation itself preserves the more detailed category structure.

The scope design is deliberately overlapping. All 91 questions carry LLM scope because the target domain is LLM-enabled applications. However, many questions also apply to traditional application surfaces: 62 questions additionally carry API scope, and 51 additionally carry Web scope. The questionnaire therefore does not partition the system into isolated LLM, Web, and API sections; instead, it models an LLM-centered application whose risks frequently overlap with Web and API concerns. This overlapping scope structure is a direct, structural expression of the unified attack-surface approach adopted throughout the thesis.

The questionnaire is adaptive: the user does not answer all questions in a fixed linear sequence, but the flow engine uses earlier answers to decide which follow-up ques-

tions are relevant. For example, public user access can trigger additional questions about authentication, authorization, abuse controls, and exposed interfaces; RAG usage can trigger additional questions about retrieval sources, document integrity, indexing, and trust in retrieved content; and tool-calling or external API access can trigger questions about tool permissions, business actions, outbound restrictions, and API authorization. This design allows the questionnaire to collect detailed information when a feature is present without forcing irrelevant questions when it is absent.

The adaptive flow is implemented separately from the question database. The question database defines the content and metadata of each question, while the flow definition defines navigation logic. The flow engine supports branching conditions such as equality, membership in a set of answers, absence of a selected option, and negative matching. During the survey it normalizes question identifiers, evaluates branching rules, determines the next unanswered question, and maintains the active path. If a user changes an earlier answer, answers that no longer belong to the active path are removed from the active response, which prevents stale answers from inactive branches from affecting the generated DFD or risk analysis. For instance, if a user first indicates that RAG is used and later changes that answer, the previously answered RAG-specific questions no longer contribute to the final model.

Several questions are designed to support deterministic DFD generation by capturing architecture-relevant signals: who can interact with the system, which interfaces expose the LLM, what preprocessing occurs before model invocation, whether RAG is used, what data sources are connected, what tools or APIs the model can access, what external services are involved, how outputs are consumed, and where security boundaries exist. One especially important question is Q85, which acts as the primary component-inventory source for the static DFD mapper; it seeds the canonical set of nodes. Other answers then refine or supplement this inventory by adding evidence about actors, entry points, data stores, model hosting, controls, and data-flow properties. When Q85 is absent or answered as unknown, the mapper falls back to inference over other questions, so the diagram is still produced from the remaining evidence rather than failing.

Beyond architecture extraction, the answers also drive deterministic candidate risk mapping. The system uses answer-derived predicates to determine which OWASP LLM, Web, and API risk categories are relevant to the target application. For example, public exposure, REST API access, file uploads, RAG usage, sensitive-data processing, tool execution, unsafe output rendering, weak authorization, unclear transport security, or missing monitoring can each activate different candidate

risks. These candidate risks are not automatically treated as final confirmed threats; rather, they define a grounded risk space that later stages can analyze and validate.

The same metadata additionally supports DREAD-based risk scoring. Each question records which DREAD dimensions it can influence. Damage is affected by signals such as sensitive data, business-critical operations, tool permissions, and potential impact. Reproducibility is affected by whether the system exposes repeatable interfaces or can be accessed by broad user groups. Exploitability is affected by input handling, exposed APIs, tool access, weak validation, and missing controls. Affected Users is affected by user population, data scope, tenant isolation, and system reach. Discoverability is affected by public interfaces, predictable endpoints, exposed application surfaces, and the visibility of attack paths.

A key design decision is that the questionnaire does not ask the user to directly estimate risk severity. Direct severity estimation would introduce subjectivity, because different users may interpret labels such as low, medium, or high risk differently. Instead, the system asks about observable system properties and control states: the user can usually answer whether the system accepts uploaded files, whether anonymous users can access it, whether authorization is checked per action, whether outputs are rendered as HTML, whether the LLM can trigger business actions, or whether logs capture suspicious activity. The system then translates these concrete answers into risk signals through deterministic logic.

The questionnaire captures both attack surface and control posture. Attack-surface questions describe what the system exposes – users, interfaces, data sources, APIs, tools, external services, model access, and output channels – while control-posture questions describe how the system is protected: authentication, authorization, input validation, output validation, rate limiting, logging, monitoring, secret management, transport security, dependency tracking, and incident response. This distinction matters because risk depends not only on the presence of a feature but also on the controls surrounding that feature. A public LLM application with tool access and weak authorization represents a different risk profile from a similar application with strict per-action authorization, rate limiting, monitoring, and output validation.

When the questionnaire is completed, the system saves the response as a structured artifact. The canonical representation is a compact map, `answers_by_flow_id`, in which each active flow question identifier is mapped to the selected answer. A detailed answer list is also stored, containing, for each entry, the flow identifier, the question identifier, the source question identifier, the question text, and the answer. Downstream services use the compact map because it provides stable, machine-

readable access to the answers, while the detailed list supports inspection and auditability.

The questionnaire therefore plays multiple methodological roles simultaneously: it is the user-facing mechanism for eliciting information, the controlled input schema for deterministic analysis, the evidence source for candidate risk mapping, the signal source for DREAD scoring, and the grounding context for LLM prompts. Its fixed structure supports reproducibility, while its adaptive flow keeps the interaction relevant to the system being modeled. By grounding the rest of the pipeline in structured questionnaire answers, the system avoids treating the LLM as the primary source of truth and instead invokes it only after the application context has been explicitly captured.

3.3 Deterministic DFD Generation

The deterministic DFD generation stage transforms the completed questionnaire response into a structured graph representation of the target application. This graph becomes one of the main grounding artifacts for later stages of the pipeline, since risks, LLM-generated threat descriptions, validation checks, and mitigations can all be related back to concrete system components and flows. The objective of this stage is not to produce a visually decorative diagram, but to derive a reproducible architectural model from the questionnaire answers.

The DFD generator is implemented as a static mapper. Its input is the saved questionnaire response, especially the canonical answer map keyed by question identifiers. The mapper accepts several compatible input shapes, including a flat map of question identifiers, the saved `answers_by_flow_id` format, and the detailed saved answer-list format. Regardless of the input shape, the first step is normalization. Question identifiers are converted into a consistent Q-number format, empty answers are ignored, and the mapper works over a normalized answer dictionary. This allows the mapper to support different response representations while preserving deterministic behavior.

After normalization, the mapper extracts architecture signals from the questionnaire answers. These signals form an intermediate representation between raw answers and graph construction. They include actors, entry points, preprocess-

ing components, RAG usage, backend processes, data stores, tools, business-action components, external services, trust boundaries, logging components, model-hosting choices, edge metadata, warnings, and assumptions. This signal layer is methodologically important because it separates the interpretation of questionnaire answers from the creation of nodes and edges. The mapper first identifies which architectural concepts are supported by the answers, and only then serializes those concepts into a graph.

A central feature of the mapper is the use of Q85 as the primary component-inventory question. Q85 provides the most direct indication of which architectural components exist in the target system. It can identify components such as a web frontend, API gateway, backend API, LLM orchestrator, RAG retriever, vector database, application database, file storage, cloud storage, tool execution runtime, external model provider, and logging or monitoring pipeline. When Q85 is answered, it seeds the canonical node set. Other questionnaire answers then refine or supplement these canonical nodes by adding evidence, metadata, or additional properties. This prevents scattered answers from creating conflicting representations of the same component.

The mapper also supports fallback inference when Q85 is absent or incomplete. In that case, architectural elements are inferred from other answers. For example, answers about direct users and exposed interfaces can imply actors and entry points; RAG-related answers can imply a RAG orchestrator and retrieval stores; tool-calling answers can imply a tool layer and tool-specific nodes; model-hosting answers can distinguish between an external provider and a self-hosted runtime; and logging answers can imply a logging and monitoring component. When the mapper relies on such inference, the graph metadata records warnings or assumptions so that the generated DFD remains auditable rather than silently presenting inferred structure as directly provided inventory.

The node-generation stage converts architecture signals into graph nodes. These nodes represent the major structural roles of the application, such as users or external systems, entry interfaces, backend processes, LLM components, data stores, tool layers, business-action components, external services, logging components, and trust boundaries. The generated graph is serialized in a node-and-edge JSON format that can be rendered by the application interface, but the methodological significance lies in the graph structure itself rather than in the frontend library used to display it. Each node carries metadata such as its label, node type, role, source, and supporting questionnaire evidence.

The mapper deliberately distinguishes between architectural components and security controls. Some answers describe controls, safeguards, or governance mechanisms rather than standalone system components. For example, prompt-injection safeguards, input filtering, output validation, approval requirements, tenant isolation, and human-review controls should not automatically become separate DFD nodes. Instead, they are attached as metadata to existing nodes when an appropriate target exists. If a control refers to a component that is not present in the graph, it is recorded as an unresolved control target in the graph metadata. This design keeps the DFD focused on architecture while preserving control evidence for later risk analysis.

Edge generation is performed after the node set has been constructed. The mapper creates flows only between existing nodes and only when the questionnaire evidence supports the relationship. Actor-to-entry edges represent interactions such as user prompts, authenticated requests, administrative actions, or integration requests. Internal edges represent request processing, orchestration, RAG retrieval, model invocation, tool execution, business API calls, responses, and logging events. The exact set of edges depends on the available nodes and the evidence extracted from the answers. This prevents the mapper from assuming a universal application topology and allows different questionnaire responses to produce different DFD structures.

The mapper applies pruning rules to remove invalid or duplicated flows. Every edge must reference nodes that exist in the graph. Duplicate edges with the same source, target, and label are removed. Request and response edges are also checked against valid actor-entry relationships. For example, a public user is not connected to an administrator panel unless the answers support that relationship. These pruning rules make the generated DFD conservative: the graph should represent supported flows rather than all theoretically possible flows.

After edge construction, the mapper enriches edges with security-relevant metadata. This metadata can include evidence questions, source questions, flow direction, trust-boundary crossing, transport-security state, sensitive-data movement, required controls, data categories, and combined-risk indicators. For example, if questionnaire answers indicate that sensitive data moves across a flow where transport protection is unclear, the corresponding edge can be marked with a combined risk indicator. This metadata does not necessarily change the visual structure of the graph, but it makes the DFD more useful for later threat analysis because flows carry security context as well as connectivity.

Trust boundaries are represented through two related but distinct mechanisms.

First, the mapper can generate explicit trust-boundary nodes, such as the public-internet boundary, the web-to-internal-API boundary, the internal-API-to-model-service boundary, a tenant-isolation boundary, an external-provider boundary, and an external-API boundary. These boundary nodes make major architectural separation points visible in the generated graph when the questionnaire evidence supports them. Second, the mapper assigns nodes to broader trust zones, such as public internet, application/backend, AI/model processing, data storage, external services, and admin/operations. These trust zones are used to detect whether an edge crosses from one security context into another. Boundary containment lists are cleaned so that they include only nodes that actually exist in the graph, and edges can be marked as crossing a trust boundary. This distinction allows the DFD to represent explicit boundary objects while also supporting systematic boundary-crossing analysis at the edge level.

The graph layout is deterministic. Nodes are assigned to stable lanes according to their role, such as actor, entry point, backend process, LLM component, data store, external service, trust boundary, or operational component. Within each lane, nodes are sorted by identifier and assigned fixed coordinate offsets. Therefore, the same questionnaire response produces not only the same nodes and edges but also the same layout. This property is useful for reproducibility, comparison, and visual inspection.

The final output of this stage is a canonical DFD artifact. It contains nodes, edges, and metadata describing the mapper version, graph mode, normalized answer count, warnings, assumptions, controls, unresolved control targets, transport-security summary, sensitive-data movement summary, and signal summary. The metadata also records orphan nodes, meaning non-boundary nodes that are not connected to any edge and therefore require inspection. The pipeline stores the generated graph as `dfd_reactflow.json` and marks it as produced by the static DFD mapper.

The methodological value of this stage is that every generated structural element remains traceable to questionnaire evidence or to an explicit mapper assumption. A component is not merely asserted; it is derived from a question answer or from a documented inference rule. A flow is not merely drawn; it is constructed between supported nodes and pruned if it is inconsistent with the answer-derived architecture. The resulting DFD is therefore a reproducible architectural baseline for the following stages: candidate risk mapping, LLM threat identification, grounding validation, and risk scoring.

3.4 Candidate Risk Mapping

After the DFD has been generated, the next stage is deterministic candidate risk mapping. The purpose of this stage is to derive a bounded and evidence-linked risk space from the questionnaire answers before LLM-based threat identification occurs. The output of this stage is not the final set of confirmed threats. Instead, it is a set of candidate risk families that are considered relevant to the target application based on its architecture, exposure, data flows, capabilities, and control posture.

The system uses a centralized candidate-risk catalog. This catalog contains 29 canonical risk codes across the three security surfaces modeled by the thesis: 10 LLM risk codes, 9 Web risk codes, and 10 API risk codes. Each catalog entry consists of a risk code, a framework label, an applicability predicate, and a set of possible evidence questions. The applicability predicate examines the normalized questionnaire answers and decides whether the corresponding risk family is in scope. This design keeps the questionnaire DREAD-oriented while placing risk-family discovery in a dedicated deterministic catalog.

This catalog-based approach replaces an earlier style in which OWASP tags were attached directly to individual questions. In the current design, questions do not themselves carry OWASP risk arrays. Instead, the catalog interprets combinations of answers. This is methodologically stronger because a risk is often implied by a pattern across several answers rather than by a single question. For example, public exposure, REST API access, tool-calling capability, weak authorization, unsafe output rendering, or RAG usage may interact to make a risk relevant. Candidate mapping therefore operates over the answer-derived system context rather than over isolated question labels.

Before candidate mapping is performed, the risk analysis service applies answer normalization. This normalization is distinct from the flow-level trimming described in the questionnaire section. Flow-level trimming removes answers that no longer belong to the active adaptive-questionnaire path. Risk-analysis normalization operates later, on the already collected answers, and resolves local inconsistencies. For example, if a multi-choice answer contains both a concrete option and an exclusive marker such as “None,” “Unknown,” or “No RAG,” the concrete option is preserved and the exclusive marker is removed. This makes the candidate mapping stage more robust to inconsistent answer combinations while retaining usable evidence.

The normalized answers are indexed by question number so that catalog predicates can inspect them consistently. Several helper predicates capture recurring architectural and security conditions. A web surface can be inferred from web chat interfaces, public dashboards, widgets, webhooks, public users, or component-inventory answers. An API surface can be inferred from REST endpoints, service-to-service calls, backend API components, internal API tools, or third-party integrations. Tool presence can be inferred from tool-use questions, agentic workflow answers, internal API access, database access, or state-changing action capabilities. RAG usage, output consumption, external API access, sensitive data handling, risky input formats, and secrets exposure are similarly inferred from answer patterns.

This predicate-based design supports the unified attack-surface model of the thesis. A candidate risk is activated because the target system exhibits a relevant architectural or control condition, not merely because the user answered a question from a particular category. RAG usage can activate risks related to data poisoning or vector-store weaknesses. Sensitive-data processing can activate information-disclosure risks. Tool execution and business-action capability can activate excessive-agency and API authorization risks. Public access and exposed interfaces can activate web and API risks. Unsafe output rendering can activate output-handling and injection-related risks. The mapping stage therefore connects concrete system properties to risk families.

The candidate discovery function returns risk families without DREAD scores or final severity levels. Each candidate contains the risk code, risk name, framework label, affected assets, missing-information notes, and questionnaire evidence. Severity is intentionally left to a later stage. In the target pipeline, candidate risks are first passed to LLM-based threat identification and grounding validation; only validated primary threats are then scored with deterministic DREAD. This separation keeps risk discovery, threat confirmation, and risk prioritization as distinct methodological steps.

Evidence filtering is central to this stage. Each catalog entry defines possible evidence questions, but a candidate risk only cites evidence questions that are actually present in the user's response. If no grounded evidence remains, the candidate is dropped. This is especially important because the questionnaire is adaptive: not every user is asked every question, and inactive branches should not contribute evidence. As a result, the candidate set is grounded in the actual response rather than in the full theoretical questionnaire.

When a DFD is available, candidate risks can also be associated with affected as-

sets. These affected assets connect abstract risk families to concrete graph elements such as entry points, LLM components, RAG components, data stores, tool layers, external services, or business-action components. This asset association does not confirm that a threat has occurred; rather, it provides structural context for later analysis. The LLM threat-identification stage can then reason over candidate risks in relation to real DFD nodes and edges, and the grounding validator can check whether generated references remain valid.

The framework label attached to each candidate is used for organization and reporting. LLM risks are grouped under the LLM framework, Web risks under the Web framework, and API risks under the API framework. These labels help structure the output and make the combined attack surface readable. However, they do not determine severity. The system uses OWASP-style codes as risk-family identifiers, while DREAD is used later for scoring. This avoids conflating framework membership with risk priority.

The mapping stage also establishes a boundary for the following LLM-based stage. Since the candidate set is selected before the LLM is invoked, it can be used as an allowlist for primary risk identification. If the model later produces a risk code outside this deterministic candidate set, that output can be rejected, demoted, or treated as secondary rather than becoming an authoritative primary risk. This point is developed further in the grounding-validation section, but its foundation is created here: candidate risk mapping defines the admissible primary risk space.

Overall, candidate risk mapping converts questionnaire evidence into a bounded, traceable, and framework-organized risk space. The DFD describes the structure of the system; candidate mapping identifies which risk families are relevant to that structure and to the answer-derived control context. Together, deterministic DFD generation and candidate risk mapping establish the architecture and risk boundaries within which LLM-based threat identification, grounding validation, DREAD scoring, and mitigation generation operate.

3.5 Local LLM-Based Threat Identification

After the deterministic identification of candidate risks, the proposed methodology introduces a local large language model as a constrained threat-identification com-

ponent. The purpose of this stage is not to replace the deterministic pipeline or to generate a threat model independently. Instead, the local LLM is used to interpret the deterministic candidate risks in the context of the specific system described by the questionnaire and the generated data-flow diagram. In this role, the model acts as a contextual reasoning layer: it enriches already selected candidate risks with system-specific threat descriptions, supporting evidence, affected architectural elements, possible abuse paths, confidence labels, and missing or weak controls.

This design deliberately separates risk selection from threat interpretation. The deterministic layer defines the initial risk universe by mapping questionnaire answers to candidate risk categories across LLM, Web, and API security concerns. The local LLM is then asked to examine this candidate set and determine how each candidate may manifest in the described system. This distinction is methodologically important because it prevents the LLM from becoming the authority on which primary risks exist. The LLM may explain, contextualize, or classify a candidate as confirmed, plausible, requiring more information, or not applicable, but it does not create the primary risk set.

The input to the local LLM is provided as a structured representation of the system. It includes the relevant questionnaire answers, the deterministic candidate risks, the generated data-flow diagram, and a fixed set of threat-pattern lenses. The data-flow diagram supplies the architectural context by listing the system elements and data flows generated from the questionnaire. The candidate risks supply the bounded security scope. The questionnaire answers provide the evidence base from which the model must reason. The model is therefore not prompted with an open-ended request such as “find all threats”; it is prompted with a bounded analytical task over a predefined set of candidate risks.

A central feature of this stage is the use of generic threat patterns as investigation lenses. These patterns are not treated as additional risk categories and do not expand the primary risk universe. Rather, they guide the model in asking how a deterministic candidate risk could appear in the concrete system. For example, a candidate related to prompt injection may be examined through a prompt-context manipulation lens; a candidate related to insecure API exposure may be examined through an authentication or authorization lens; and a candidate related to retrieval-augmented generation may be examined through a retrieval or memory contamination lens. The patterns provide analytical structure while keeping the final primary findings tied to the deterministic candidate set.

The LLM output is constrained through a structured response format. Each pri-

Table 3.1 Threat-pattern lenses used during local LLM-based threat identification

Threat-pattern lens	**Methodological role**
Prompt-context manipulation	Checks whether untrusted text can influence model context, instructions, or output behavior.
Trust-boundary input crossing	Checks whether lower-trust input crosses a trust boundary without sufficient validation.
RAG or memory contamination	Checks whether retrieved, indexed, or remembered content can be poisoned or reused unsafely.
Sensitive-data exposure	Checks whether sensitive data can be returned, retained, logged, or exposed.
Excessive tool or workflow agency	Checks whether the model can trigger tools or business actions without adequate control.
Weak authentication or authorization	Checks authentication, authorization, and object/function-level access-control weaknesses.
Unsafe output handling	Checks whether model output is consumed downstream without validation or sanitization.
Vector-store or embedding isolation	Checks whether vector stores or embeddings lack tenant or user isolation.
Secrets, logging, and transport exposure	Checks whether secrets, logs, or transport paths expose sensitive information.
Missing monitoring, limits, and incident response	Checks whether rate limits, monitoring, alerting, or incident-response controls are missing.

mary threat must correspond to one of the deterministic candidate risk categories supplied to the model. References to affected architectural elements must be selected from the elements that actually exist in the generated data-flow diagram. The allowed status values are also fixed: a finding may be confirmed, plausible, require more information, or be marked as not applicable. These restrictions are implemented through a structured JSON schema with enumerated values for candidate risk categories, DFD references, and status labels. In other words, the model is not only instructed to stay within the allowed boundary; the response format itself constrains the values that can be returned. Any residual inconsistency is handled by the deterministic grounding-validation stage described in the next section.

For each candidate risk, the model is required to produce one structured finding. A finding contains the candidate risk category, a threat name, a status label, the threat-pattern lens used, supporting evidence, affected architectural elements, a possible abuse path, a control gap, a confidence label, and any missing information. The abuse path is especially important because it forces the model to describe how an attacker would move from an entry point or precondition to a security impact within the specific system. The control gap is equally important because it identifies the missing, weak, or unclear protection that allows the threat to remain relevant.

The status labels reflect evidence strength. A confirmed threat requires specific evidence and a clear control gap. A plausible threat is supported by the available system description but still involves some uncertainty or assumption. A finding that requires more information is used when the questionnaire answers or DFD context do not provide enough grounding. A not-applicable finding is used when the candidate risk does not appear to apply to the described system. This status distinction prevents the model from treating every candidate as an equally strong finding and supports a more transparent interpretation of uncertainty.

To avoid candidate omission, the model is instructed to address every candidate risk in its assigned input set. If a candidate cannot be confidently grounded, it must still be represented with an appropriate status such as requiring more information or not applicable. This is important because omission would make it unclear whether the model considered the candidate and rejected it, failed to analyze it, or simply exceeded its attention capacity. Explicit coverage makes the output more auditable.

In addition to the primary candidate analysis, the methodology includes a limited secondary scan. During this scan, the LLM may report a small number of material observations that appear relevant to the described system but do not fit the deterministic candidate set. This secondary scan is intentionally disciplined rather

than open-ended. Such observations must be grounded in the questionnaire or DFD context, must be specific to the system, and must remain advisory. They are not promoted into primary threats, are not assigned final severity, and do not participate in deterministic DREAD scoring. Their role is to preserve potentially useful out-of-set signals without weakening the deterministic-first boundary.

This secondary channel is important because no deterministic catalogue can guarantee perfect coverage of every possible system-specific security concern. However, allowing unrestricted LLM discovery would undermine the reproducibility and grounding goals of the methodology. The secondary scan therefore represents a compromise: the model is allowed to surface high-signal out-of-set concerns, but those concerns remain explicitly separated from the authoritative scored threat model. They are treated as prompts for further analysis rather than as validated findings.

Candidate risks may be processed in smaller groups rather than all at once. This chunking strategy reduces the likelihood that the local model ignores some candidates when the candidate set is large. For a fixed input order and fixed group size, the grouping strategy itself is deterministic. However, the content generated by the LLM may still vary across repeated executions, even when decoding parameters are fixed. For this reason, this stage is not treated as fully reproducible in the same sense as the deterministic DFD, candidate-risk mapping, or DREAD scoring stages. Instead, its contribution is evaluated through structural validity, grounding, specificity, and variance over repeated runs.

The execution status of the LLM stage is also recorded. If all groups are processed successfully, the stage is considered completed. If only some groups are processed, the stage is considered partial, and the number of successful groups is retained for transparency. If the local model is unavailable or produces no usable structured output, the stage is considered unavailable. In that case, the pipeline falls back to the deterministic baseline. This ensures that the LLM can enrich the threat model but cannot prevent the system from producing a valid risk analysis.

A simplified example illustrates the intended behavior. Suppose the deterministic layer identifies prompt injection as a candidate risk for a public chat-based LLM application. The LLM may classify this candidate as plausible if anonymous users can submit text that reaches the model context and the questionnaire does not describe prompt isolation or input filtering. The model would then describe an abuse path in which an attacker submits malicious instructions through the chat interface, the input is forwarded to the LLM as part of the context, and the model may follow or reveal information contrary to the system's intended policy. The

control gap would be the absence or weakness of prompt/content isolation. The important point is that the LLM is explaining how the deterministic candidate manifests in this system, not inventing the candidate itself.

This stage therefore contributes contextual depth rather than authoritative scoring. It turns deterministic candidate risks into richer, system-specific threat descriptions, while preserving the deterministic pipeline's control over the primary risk universe, architectural model, and severity calculation. The local LLM is valuable because it can articulate abuse paths and control gaps that would be difficult to express through static templates alone. At the same time, the surrounding constraints prevent it from becoming an unconstrained threat generator.

3.6 Grounding Validation and Hallucination Control

All LLM-generated threat-identification output is passed through a deterministic grounding-validation stage before it can influence scoring. This stage is necessary because the methodology treats the LLM output as useful but not authoritative. The validator checks whether the model's findings remain inside the deterministic candidate-risk boundary, whether architectural references correspond to real DFD elements, and whether the stated evidence is sufficient for the assigned status. Only findings that pass these checks are allowed to proceed as validated primary threats.

The validator relies on three authoritative sources: the deterministic candidate-risk set, the generated data-flow diagram, and the questionnaire-derived evidence associated with each candidate. The candidate-risk set defines which risks may become primary scored threats. The data-flow diagram defines which components, actors, data stores, trust boundaries, and flows may be referenced. The questionnaire evidence defines what is actually known about the system. By comparing the LLM output against these sources, the validator converts grounding from a prompt instruction into an enforceable methodological control.

A distinction is made between demotion and downgrading. Demotion means that a finding is moved out of the primary threat set because it violates the deterministic candidate-risk boundary. Downgrading means that a finding remains associated with a deterministic candidate but its status is reduced because the evidence is not strong enough. For example, an out-of-set risk category is demoted to an advisory

finding, while a weakly supported confirmed finding is downgraded to plausible or requiring more information.

Table 3.2 Grounding-validation rules for hallucination control

Condition detected during validation	**Validator action**	**Methodological purpose**
The LLM uses a primary risk category that was not selected by the deterministic layer.	Move the finding to the advisory secondary set and mark it as requiring more information.	Prevents the LLM from expanding the scored primary risk universe.
The LLM references a non-existent architectural element or data flow.	Remove the invalid reference and record the correction.	Prevents hallucinated DFD elements from supporting a threat.
The finding has no supporting evidence and no stated control gap.	Downgrade the finding to requiring more information.	Prevents unsupported findings from being treated as grounded.
The underlying questionnaire evidence is unknown or absent.	Prevent the finding from remaining confirmed.	Prevents unknowns from being converted into certainty.
A confirmed finding has weak support or low confidence.	Downgrade the finding to plausible or requiring more information.	Prevents overconfident threat classification.
A deterministic candidate is omitted by the LLM.	Report it as an unaddressed deterministic candidate.	Prevents candidate risks from silently disappearing.

These rules protect the two main boundaries of the methodology. The first boundary is the primary risk boundary: only deterministic candidates can become validated primary threats. The second boundary is the architectural grounding boundary: only real DFD elements can support a threat. Together, these rules prevent the LLM from introducing unsupported risk categories, relying on invented system structure, or overstating uncertain evidence.

A short example demonstrates the validation logic. Suppose the deterministic candidate set contains prompt injection and sensitive-information disclosure, but the LLM returns an additional primary finding for an unrelated API risk that was not selected by the deterministic layer. The validator does not allow this additional finding to remain primary. It moves it to the advisory set and marks it as requiring more information. If the same finding also references a component that does not exist in the generated DFD, that reference is removed and recorded. If a candidate is marked as confirmed while its evidence is unknown or weak, its status is reduced

accordingly. The result is that the model's observations may remain visible for transparency, but they cannot influence scoring unless they satisfy the deterministic grounding rules.

The validator also prevents silent omission. If the deterministic layer selected a candidate risk but the LLM failed to address it, the candidate is reported as unaddressed rather than disappearing from the analysis. This is important because the absence of an LLM finding should not be interpreted as evidence that the deterministic candidate was irrelevant. In the proposed methodology, omitted candidates are retained as transparent uncertainty rather than removed from the record.

After validation, findings are separated into four conceptual groups: validated primary threats, downgraded findings, advisory secondary findings, and unaddressed deterministic candidates. Only validated primary threats proceed to deterministic DREAD scoring. Downgraded findings, advisory secondary findings, and unaddressed candidates remain available for transparency, but they do not determine final severity. This separation allows the system to preserve useful model output while maintaining a strict boundary around scored results.

The audit records produced by grounding validation are directly connected to the evaluation of hallucination control. The number of non-candidate risk categories demoted measures attempts by the LLM to exceed the deterministic risk boundary. The number of invalid architectural references removed measures hallucinated DFD usage. The number of unsupported confirmed findings downgraded measures overclaiming. The number of unaddressed deterministic candidates measures omission. These metrics provide an automatic basis for evaluating guardrail effectiveness in relation to the research question on hallucination control.

Since DREAD scoring occurs only after deterministic grounding validation, the LLM cannot determine final risk severity. It cannot promote an unsupported finding into a scored threat, cannot use non-existent architectural elements as evidence, and cannot introduce new primary risk categories outside the deterministic candidate set. The validator is therefore the methodological control point between generative enrichment and authoritative risk assessment. Overall, this stage allows the methodology to benefit from the explanatory strengths of a local LLM while controlling hallucination, omission, and overconfidence risks.

3.7 Deterministic DREAD-Based Risk Scoring

After threat identification and grounding validation, the methodology applies deterministic DREAD-based risk scoring to the validated primary threats. This stage is intentionally placed after grounding validation so that only threats with sufficient structural and evidential support can influence the final risk model. Findings that are advisory, downgraded to requiring more information, marked as not applicable, or backfilled as unaddressed candidates do not proceed to scoring. This preserves the distinction between observed uncertainty and scored risk.

The scoring model is based on the five DREAD dimensions: Damage, Reproducibility, Exploitability, Affected Users, and Discoverability. Each dimension is assigned a value from 1 to 3, where 1 represents a lower contribution to risk and 3 represents a higher contribution. The five dimension scores are summed to produce a total score between 5 and 15. The average score is then calculated by dividing the total by five. The final risk level is derived from this average rather than from the LLM or from manually assigned severity labels.

The risk level is derived from the DREAD average using fixed cut-offs, as shown in Table 3.3.

Table 3.3 DREAD average cut-offs used for deterministic risk levels

DREAD average	**Equivalent total**	**Risk level**
≥ 2.7	14–15	Critical
2.2–2.69	11–13	High
1.5–2.19	8–10	Medium
< 1.5	5–7	Low

Because the level is a function of the average alone, it is fully determined by the five dimension scores and involves no manual or model-assigned severity.

This scoring stage is fully deterministic. Given the same questionnaire answers and the same validated threat set, the same DREAD scores and risk levels are produced. The LLM does not compute DREAD dimensions, does not assign the final risk level, and cannot modify the score. This is a key methodological boundary: the LLM may contribute contextual threat interpretation, but risk prioritization remains reproducible and auditable.

The DREAD score is derived from questionnaire-grounded signals. Each scoring rule reads specific answer-derived properties of the system, such as whether the application is public-facing, whether it processes sensitive data, whether the LLM can trigger state-changing actions, whether tenant isolation is present, whether input parsing is strict, whether replay protections exist, and whether adversarial testing or incident response processes are in place. These signals allow the scoring model to reflect system context rather than assigning generic severity to every OWASP category.

Damage is treated as partly risk-specific. Different risk categories are driven by different kinds of impact. Data-exposure risks are affected by the presence of sensitive data, credentials, retention, and regulatory impact. Agency-related risks are affected by whether the LLM can execute workflows, modify system state, send emails, approve transactions, or trigger business-critical actions. Availability risks are affected by rate limits, quotas, resource caps, and abuse protections. Integrity risks are affected by shared retrieval stores, poisoning exposure, and whether untrusted content can influence other users. Access-control risks are affected by the combination of sensitive data and privileged operations.

The Damage score is also bounded by broader business context. If the questionnaire indicates severe business, financial, legal, or regulatory impact (Q87), the damage score may be raised. If the system retains prompts, responses, logs, memory, or vector data without clear deletion controls (Q88), data-related damage may remain elevated. Conversely, mature incident response or containment processes (Q89), or explicitly low operational impact (Q87), can reduce or cap the realized damage. This prevents the score from depending only on technical exposure while ignoring business consequences.

Reproducibility measures how consistently an attacker could repeat the threat. It is influenced by the presence or absence of controls specific to the risk category, such as prompt-injection safeguards, output validation, access controls, rate limits, review processes, or tool-approval controls. It is also affected by replay behavior and adversarial testing maturity. If malicious inputs or state-changing actions can be replayed without strong controls (Q91), reproducibility increases. If replay is blocked or requires fresh authorization (Q91), reproducibility decreases. Mature adversarial testing and regression testing (Q90) also reduce reproducibility because recurring exploit paths are more likely to be detected and closed.

Exploitability measures how easily an attacker can reach and exercise the weakness. It is influenced by who can access the system (Q2), whether authentication

is required (Q25), whether the surface is public or internal, and whether input defenses exist. Public anonymous access increases exploitability, while internal-only or administrator-only access lowers it. Input format and parsing behavior also matter: support for complex formats such as HTML, documents, code, structured data, or multimodal input (Q83) increases the attack surface when parsing and normalization are weak (Q84). Strict validation and normalization (Q84) reduce exploitability.

Affected Users measures blast radius. It reflects tenancy, isolation, shared stores, memory boundaries, and user or tenant scale. Multi-tenant deployment and weak or unclear tenant isolation (Q40) raise the score, while strong per-tenant isolation lowers it. Shared indexes, shared caches, or memory that crosses user boundaries also increase this score. Public internet-scale or multi-tenant use (Q86) can raise the minimum affected-user score, whereas single-user or local-only use can lower it when no shared stores are involved.

Discoverability measures how easily an attacker can find, probe, or infer the weakness. Detailed internal errors (Q76), undocumented or debug APIs (Q59), and public exposure (Q2) increase discoverability, whereas sanitized error handling combined with security monitoring (Q33) reduces it. Adversarial testing and incident-response maturity (Q89 and Q90) can also reduce discoverability because weaknesses are more likely to be identified and closed before external probing.

The scoring model also records which DREAD dimensions are strongest for each risk. This is useful because two risks can share the same final band while being driven by different causes. For example, one High risk may be driven primarily by Damage and Affected Users, while another may be driven by Exploitability and Discoverability. Recording the strongest dimensions improves interpretability and supports more targeted mitigation generation in the next stage.

The output of this stage includes the DREAD dimension scores, total score, average score, final risk band, and the strongest contributing dimensions. The final risk list is then ordered by severity, score, and risk category. Because all scores are derived from questionnaire answers and fixed scoring rules, this stage supports the reproducibility claim of the methodology. It also makes the final prioritization auditable: each score can be traced back to concrete answer-derived signals rather than to opaque model judgment.

DREAD is deliberately a coarse, triage-oriented risk model: each dimension uses a small ordinal scale from 1 to 3 derived from observable, yes/no-style system properties rather than fine-grained expert judgment. This coarseness is intentional, as

it enables deterministic, reproducible, and questionnaire-driven scoring without requiring the analyst to estimate severity directly. Within this model, only Damage and Reproducibility are scored per risk category; Exploitability, Affected Users, and Discoverability are derived from system-level exposure signals and are therefore shared across all codes analysed for a given system. This is a deliberate simplification consistent with the questionnaire-driven design, but it limits inter-risk differentiation within a single system: because the shared dimensions shift all codes together, many risks in the same system may converge on the same band. This inherent discrimination limit is acknowledged and discussed further in the limitations of the approach.

3.8 Mitigation Generation

After validated threats have been scored through deterministic DREAD, the pipeline generates mitigations. This stage is deliberately placed after scoring so that mitigation recommendations are grounded in threats that have already passed validation and have already received a deterministic risk level. The mitigation stage does not identify new risks, does not change DREAD scores, and does not alter final severity. Its role is to propose actionable controls for risks that are already part of the validated threat model.

The methodology combines deterministic mitigation guidance with optional local LLM-based mitigation enrichment. The deterministic layer provides stable baseline controls for known risk categories, drawn from a static catalogue of category-level mitigations combined with DREAD-aware mitigation logic. These controls ensure that every scored risk can receive at least generic but relevant mitigation advice even if the local model is unavailable. This preserves the same fallback principle used elsewhere in the pipeline: the LLM can improve specificity, but the system does not depend on it to produce a valid output.

In addition to generic category-based controls, the deterministic layer also uses DREAD-aware mitigation logic. This means that recommendations are influenced not only by the risk category but also by the DREAD dimensions that contributed most strongly to the score. For example, if Damage is high because of severe business impact or long-term retention, the recommendations emphasize approval controls, segregation of duties, transaction limits, retention limits, and deletion controls. If

Reproducibility is high because replay is possible or adversarial testing is absent, the recommendations emphasize replay protection, fresh authorization, and regression testing. If Affected Users is high because of public or multi-tenant scale, the recommendations emphasize tenant isolation and blast-radius reduction.

Similarly, if Exploitability is high because risky input formats are accepted with weak parsing, the recommendations emphasize strict parsers, file validation, sandboxing, content normalization, and instruction/data isolation. If Discoverability is high, the recommendations emphasize reducing exposure, adding authentication, rate limiting, monitoring, and alerting. This DREAD-aware approach makes mitigation generation more closely tied to the reasons behind the score rather than merely attaching a static checklist to each risk category.

The optional local LLM mitigation stage receives only validated and DREAD-scored threats. It is given the fixed risk category, risk level, DREAD summary, threat status, control gap, supporting evidence, affected components, and relevant DFD context. These inputs are treated as read-only. The model is not asked to reassess the risk, discover additional threats, or modify the score. It is asked only to propose concrete, verifiable engineering actions for the supplied risks.

The mitigation output is constrained through a structured response format. Each mitigation must refer to one of the supplied risk categories. The model cannot introduce mitigations for new risk categories outside the scored threat set. Each mitigation is expected to include a title, an action, a priority, a target component where applicable, a validation step, and a link back to evidence or to the threat's control gap. Here, priority denotes the recommended implementation urgency of the mitigation and is distinct from the DREAD-derived risk level of the underlying threat; a high-severity risk may still admit a lower-priority, quick-to-deploy control, and vice versa.

The requirement that each mitigation map to evidence or a control gap is important. A mitigation should respond to a demonstrated weakness in the threat model. If the threat identifies missing prompt isolation, the mitigation should address prompt/content separation, input treatment, or instruction hierarchy. If the threat identifies weak authorization, the mitigation should address server-side authorization, least privilege, or object/function-level access checks. If a generated mitigation cannot be connected to evidence or to a control gap, it is excluded or grounded using the known control gap. This prevents the model from producing plausible but unsupported recommendations.

Mitigations are generated in small batches rather than in one large request. This is a practical design choice for local models, which may struggle with large structured outputs or long response times. Batching keeps each request bounded and allows partial success. If all batches succeed, mitigation generation is marked as completed. If some batches succeed and others fail, the stage is marked as partial and the successful mitigations are retained. If all batches fail or the model is unavailable, the stage is marked as unavailable and the deterministic mitigation guidance remains available.

The mitigation stage also produces quick wins, assumptions, and missing-information notes. Quick wins provide concise, high-impact actions for the highest-priority risks. Assumptions and missing-information notes preserve transparency about the conditions under which recommendations were generated. For example, if important information about incident response, retention, replay behavior, or user scale is missing, the final output can indicate that the mitigation advice is based on incomplete information.

A simplified example illustrates the stage. If a validated prompt-injection threat receives a High risk level because the system is public-facing, accepts untrusted input, and lacks prompt isolation, deterministic guidance may recommend isolating the system prompt, treating all user input as untrusted, and limiting the privileges of downstream actions. The LLM may then enrich this with more specific engineering steps, such as separating instruction and data channels, applying schema-based input validation, adding prompt-injection regression tests, and verifying that tool calls cannot be triggered without authorization. These actions are useful because they are tied to the threat's abuse path and control gap.

The mitigation stage therefore serves as the final translation layer from risk analysis to security action. It converts validated, scored threats into controls that can be implemented and verified. However, it remains subordinate to the deterministic-first methodology: it cannot create new scored risks, cannot change severity, and cannot override the grounding validator or DREAD model. Its contribution is specificity, actionability, and engineering detail, not risk authority.

3.9 Local LLM Integration and Deterministic Resilience

The proposed methodology integrates the local LLM as a controlled enrichment layer within a deterministic-first threat-modeling pipeline. The LLM is not used as a general threat-model generator and is not treated as the authority for architecture, candidate-risk selection, grounding, or severity. Instead, it is invoked at two specific points where natural-language reasoning can add value: threat identification and mitigation generation. During threat identification, the model contextualizes deterministic candidate risks into system-specific abuse paths, control gaps, affected elements, and evidence statements. During mitigation generation, it translates validated and scored threats into more concrete engineering actions. In both cases, the LLM operates inside boundaries established by deterministic stages.

The LLM layer is model-agnostic. The selected local model can be changed through configuration without changing the deterministic parts of the pipeline. This means that the questionnaire, DFD generation logic, candidate-risk mapping, grounding validation, and DREAD scoring rules remain fixed while different local models can be tested. This design allows the thesis to distinguish between the contribution of the pipeline architecture and the capability of a particular model. If one local model produces less specific threat descriptions or weaker mitigations, that limitation can be studied as a model-capability issue rather than as a failure of the deterministic-first methodology.

This model-agnostic structure also supports the later fine-tuning experiment. A base local model and a fine-tuned local model can be compared under the same questionnaire input, output schema, grounding rules, validation logic, and scoring method. Any improvement in schema conformance, evidence linkage, system-specificity, or mitigation actionability can therefore be evaluated while holding the deterministic pipeline constant. This is important because the thesis does not claim that one particular model is inherently sufficient for all threat-modeling tasks; rather, it investigates how a local model behaves when placed inside a constrained and auditable workflow.

In the intended local deployment, the LLM layer also reduces dependence on external cloud inference. Threat modeling may involve sensitive architectural details, exposed interfaces, authentication design, data categories, tool permissions, and operational controls. Keeping the model execution local avoids requiring these details to be sent to an external cloud model during analysis. This does not by itself make the

implementation production-secure, but it is consistent with the methodological goal of keeping the LLM layer configurable, inspectable, and under local control.

The LLM is invoked only after deterministic context has been established. During threat identification, it receives a bounded set of deterministic candidate risks, questionnaire-derived evidence, DFD context, and threat-pattern lenses. It is not asked to freely invent the complete threat model. Its primary task is to explain how each candidate may or may not manifest in the described system. A limited secondary scan is permitted for material out-of-set observations, but these observations remain advisory, are excluded from scoring, and would require separate manual review outside the automated V1 pipeline.

During mitigation generation, the LLM is invoked even later in the workflow. It receives only threats that have already been identified, grounded, validated, and scored. The model receives the risk category, DREAD-derived risk level, DREAD summary, threat status, control gap, supporting evidence, affected elements, and DFD context as read-only input. It is not asked to reassess the risk, discover additional threats, or modify severity. Its role is limited to producing concrete, verifiable mitigation actions for risks that are already part of the validated threat model.

Both LLM-supported stages use structured outputs rather than unconstrained prose. In threat identification, primary risk categories and DFD references are restricted to allowed values derived from the deterministic context. In mitigation generation, each mitigation must refer to a supplied scored risk and must connect to evidence or a control gap. This design makes the LLM output machine-checkable and allows deterministic components to reject, demote, or ignore unsupported content. The model's contribution is therefore captured as structured enrichment rather than accepted as a free-form authority.

The LLM stages are also designed to operate in bounded units. Threat identification can be divided into smaller groups of candidate risks, and mitigation generation can be divided into smaller groups of scored threats. This reduces the likelihood that a local model omits items, times out, or fails to produce a valid structured response when the input becomes large. If all units succeed, the stage is recorded as completed. If only some units succeed, the stage is recorded as partially completed and the successful outputs are retained. If the stage is skipped, disabled, or unavailable, that status is also recorded. This provenance makes the final output transparent about how much of the result was LLM-enriched.

The deterministic-resilience mechanism ensures that LLM failure does not invalidate

the overall pipeline. This is not the preferred mode of operation, since the LLM adds valuable specificity and explanatory depth. However, the system must remain capable of producing a valid and auditable threat model even if the local model is unavailable, times out, or returns unusable structured output. In such cases, deterministic components continue to provide architecture generation, candidate-risk mapping, DREAD scoring, and baseline mitigation guidance.

Table 3.4 Resilience behavior of the local LLM-supported pipeline

LLM-related condition	**Pipeline behavior**	**Validity of final output**
Local LLM threat identification is completely unavailable or produces no usable output.	The pipeline falls back to deterministic risk analysis: candidate risks are scored directly using deterministic DREAD, and deterministic mitigation guidance is attached.	The output remains valid, grounded, and reproducible, but it lacks LLM-generated abuse paths, control-gap explanations, and contextual threat-identification detail.
Local LLM mitigation generation is unavailable or partially fails after threats have already been validated and scored.	The scored threat model remains unchanged. Successful mitigation batches are layered on top when present; the deterministic mitigation guidance attached to every risk remains available in all cases.	The risk model remains valid because severity was already determined deterministically. Only mitigation specificity and actionability may be reduced.

This resilience design supports controlled comparison between deterministic and LLM-enriched outputs. The deterministic baseline represents what the system can produce from questionnaire evidence alone. The LLM-enriched path represents the additional contribution of the model under structured-output and grounding constraints. This difference is central to the evaluation: the model's contribution can be assessed in terms of system-specificity, abuse-path detail, control-gap articulation, mitigation actionability, schema validity, and grounding, without allowing the model to control the architecture or risk severity.

Overall, the local LLM layer is constrained, replaceable, and non-authoritative. It is constrained because it operates over deterministic context and structured outputs. It is replaceable because the selected local model can be changed without altering the deterministic workflow. It is non-authoritative because it cannot define the DFD, select the primary risk universe, validate its own grounding, or assign final severity. The result is a methodology in which local LLMs can contribute meaningful contex-

tual reasoning while deterministic mechanisms preserve reliability, traceability, and reproducibility.

3.10 Local LLM Fine-Tuning Methodology

The local LLM is used in two stages of the proposed pipeline: threat identification and mitigation generation. These stages require language-rich reasoning because the model must explain how a candidate risk manifests in a specific system, describe plausible abuse paths and control gaps, and generate mitigation actions linked to affected components and evidence. The motivation for fine-tuning is to improve the structure, grounding, and system-specificity of these two LLM-supported stages, not to transfer responsibility for data-flow diagram construction, candidate-risk selection, or DREAD severity scoring to the model.

This thesis considers supervised fine-tuning as the main mechanism for improving local LLM effectiveness in threat identification and mitigation generation. The training data is split by task rather than organized as a single end-to-end risk-report dataset. The threat-identification dataset is designed to teach the model to produce grounded threat descriptions, affected data-flow diagram nodes or edges, evidence, abuse paths, control gaps, status values, confidence values, and structured risk references from questionnaire-derived context, candidate OWASP Web/API/LLM risks, and data-flow diagram elements. The mitigation-generation dataset is designed to teach the model to produce concrete mitigations, priorities, target components, validation steps, and evidence mappings from validated and DREAD-scored threats.

The datasets are prepared in chat-based JSONL format with system, user, and assistant messages, matching the instruction-following format used during pipeline execution. Each example follows the same constraints used by the application: primary risk codes must come from the supplied candidate set, data-flow diagram references must correspond to real nodes or edges, and mitigation outputs must map back to supplied threats, evidence, or control gaps. This design is intended to teach the model the expected structure and grounding behavior of the pipeline, rather than allowing it to replace the deterministic stages.

Because the training data is grounded in outputs produced by the same deterministic pipeline, this design introduces a circularity risk. The fine-tuned model may

learn the pipeline's preferred structure and label distribution rather than independently discovering higher-quality security findings. For this reason, the deterministic DREAD scorer and same-distribution generated labels are not treated as an external gold standard for semantic correctness. Fine-tuning should therefore be evaluated only through structural and grounding-oriented metrics, using evaluation examples that are held out from the training data.

Two dataset characteristics are treated as limitations. First, the current threat-identification data does not include positive examples for out-of-catalog secondary threats. Fine-tuning on this data may therefore bias the model toward reporting only risks already present in the supplied candidate set, even though the pipeline allows rare, high-signal secondary findings when they are strongly supported by system evidence. Second, the confidence labels are not assumed to be calibrated probabilities. If their distribution is imbalanced, the fine-tuned model may inherit that prior, so confidence outputs should be inspected during evaluation rather than treated as independent evidence of correctness.

Low-Rank Adaptation (LoRA) is selected as the fine-tuning method because it is more cost-effective than full model fine-tuning. Instead of updating all model parameters, LoRA trains a small number of low-rank adapter parameters while keeping the base model largely unchanged. This reduces GPU memory requirements, training cost, storage overhead, and iteration time. It also fits the model-agnostic design of the pipeline: adapters can be trained, replaced, or updated for the threat-identification and mitigation-generation tasks without maintaining multiple fully fine-tuned copies of the base model.

Fine-tuning also supports the sustainability objective of this thesis. As new LLM attack patterns and mitigation practices emerge, trusted security sources can be used to create additional task-specific examples for the threat-identification and mitigation-generation datasets. These examples can then be incorporated through periodic adapter-based fine-tuning. In this way, the local LLM can be updated with new threat knowledge while the surrounding pipeline continues to enforce grounding against the questionnaire-derived data-flow diagram, candidate risk set, and deterministic DREAD scores.

The purpose of fine-tuning in this methodology is therefore not to create an unconstrained autonomous threat modeler. Rather, it is to improve the local LLM's ability to produce structured, grounded, system-specific, and evidence-linked outputs within a controlled threat-modeling workflow. Whether fine-tuning improves the measured behavior of the model is addressed in the evaluation and results chap-

ters, not assumed in the methodology.

3.11 Sustainable Threat Intelligence and Continuous Model Improvement

The final methodological component of this thesis is a planned sustainability-oriented pipeline for continuous threat-intelligence collection and model improvement. This component is separate from the main threat-modeling pipeline described in the previous sections. While the main pipeline generates a threat model from questionnaire answers, deterministic mappings, local LLM enrichment, grounding validation, DREAD scoring, and mitigation generation, the sustainability pipeline is intended to keep the system's security knowledge current over time.

This motivation is especially important for LLM-enabled applications because the LLM security landscape evolves rapidly. New attack surfaces, abuse techniques, prompt-injection variants, agentic workflow risks, retrieval-related weaknesses, tool-use vulnerabilities, and mitigation strategies continue to appear in recent academic publications, security reports, vulnerability disclosures, and trusted technical sources. Therefore, a static risk catalogue or a one-time fine-tuning dataset may gradually become outdated as new patterns emerge.

The planned sustainability pipeline would address this limitation by periodically collecting material from reliable and up-to-date sources. These sources may include peer-reviewed research papers, recent conference and workshop publications, OWASP updates, vendor security advisories, vulnerability databases, incident reports, and trusted security engineering blogs. The purpose is not to blindly scrape arbitrary web content, but to build a controlled source set from materials that are credible, relevant, and recent.

After collection, the gathered material would be filtered and normalized into structured threat-intelligence records. Each record would describe the attack pattern, the affected LLM-enabled application component, the required preconditions, the possible impact, the related control gaps, observable evidence signals, and potential mitigations. This structure is important because the proposed system depends on deterministic grounding. New knowledge should not enter the automated threat model as free-form text; it must first be transformed into a format that can be

connected to questionnaire answers, DFD elements, risk categories, and validation rules.

The output of this pipeline would also support continuous improvement of the local LLM. Once the collected attack-surface knowledge has been reviewed, cleaned, and converted into structured examples, it can be used as additional fine-tuning data. These examples could teach the local model to produce more accurate grounded threat descriptions, better evidence-linked abuse paths, more relevant control-gap explanations, and more actionable mitigation recommendations for newly emerging LLM security risks.

However, this continuous improvement process would remain subordinate to the deterministic-first methodology. The fine-tuned model would not be allowed to independently define the architecture, introduce unsupported primary risks, modify DREAD scores, or change final severity levels. Instead, the improved model would operate within the same constraints as the base model: deterministic candidate risks, DFD grounding, structured outputs, validation checks, and deterministic scoring. In this way, the system can become more current without sacrificing reproducibility, traceability, or hallucination control.

This sustainability mechanism creates a controlled feedback loop. Trusted and recent external knowledge is collected, filtered, structured, reviewed, transformed into fine-tuning examples, used to improve the local LLM, and then evaluated again under the same validation and scoring constraints. Over time, this can help the system remain aligned with the evolving LLM security field while preserving the methodological separation between deterministic authority and LLM-based enrichment.

Therefore, sustainability in this thesis refers to the long-term ability of the proposed approach to remain relevant as LLM security changes. The main implemented pipeline provides reproducible threat modeling for a given system, while the planned sustainability pipeline provides a path for updating the system's knowledge base and improving the local model as new LLM attack surfaces are discovered.

4. RESULTS

This chapter presents the observed behavior of the implemented questionnaire-driven threat modeling workflow. The results focus on structural feasibility, deterministic architecture generation, candidate-risk mapping, DREAD-based scoring behavior, and the fine-tuning diagnostics obtained for the two LLM-supported stages. Since the proposed workflow is designed around deterministic grounding, the evaluation emphasizes whether the system can consistently transform questionnaire answers into a usable threat-modeling artifact rather than whether the LLM independently discovers all possible threats.

The results are organized around the main stages of the pipeline. First, the chapter discusses the behavior of the static DFD mapper and the practical trade-off between diagram completeness and readability. Second, it describes the deterministic candidate-risk and DREAD scoring behavior. Third, it discusses the role of the local LLM as an enrichment layer. Finally, it presents the fine-tuning diagnostics for threat identification and mitigation generation using 500, 1000, and 1500 training examples.

4.1 Static DFD Mapper Behavior

The static DFD mapper was able to convert questionnaire answers into a repeatable architectural representation without relying on the LLM. This was important for the thesis because it prevented the model from inventing core system components or changing the architecture between runs. In general, the mapper produced stable nodes and flows for common application features such as web entry points, API layers, LLM orchestration, retrieval components, vector stores, external services,

logging, and monitoring.

The main observation from the mapper implementation was that deterministic DFD generation requires careful control over mapping granularity. When the mapper rules were too strict, the generated diagram sometimes omitted supporting elements such as monitoring, logging, validation, or control-related components. This reduced the visibility of security-relevant controls and made some risks harder to connect to architectural evidence. In contrast, when the rules were too permissive, the mapper tended to create overly detailed diagrams in which individual actions or minor questionnaire signals became separate nodes. This increased diagram complexity and made the output harder to interpret.

A recurring issue was the balance between completeness and readability. A security-oriented DFD must include enough components to support risk reasoning, but it should not model every questionnaire answer as a separate architectural element. The most useful outputs were obtained when the mapper represented major trust-boundary crossings, persistent data stores, externally reachable components, LLM-specific processing stages, and security-relevant control points, while keeping minor behavioral details as evidence fields rather than standalone DFD nodes.

During development, some edge cases were also observed when unusual answer combinations produced incomplete or inconsistent intermediate structures. These cases did not invalidate the deterministic design, but they showed that static architecture generation needs defensive handling for missing answers, contradictory questionnaire states, and optional components. This motivated the use of validation checks for orphan nodes, missing flows, and unsupported references before the generated DFD is used by later pipeline stages.

4.2 DFD Integrity and Orphan Node Handling

The DFD integrity checks were useful for detecting components that were created without meaningful connections to the rest of the system. Orphan nodes are problematic because they may later be referenced by risks or mitigations even though they do not participate in an explicit data flow. Strict orphan-node removal improved diagram cleanliness, but in some cases it also removed control-related elements that were security-relevant but weakly connected in the initial mapping rules.

More permissive orphan-node handling preserved these elements, but it sometimes produced diagrams with unnecessary nodes. This created a second problem: when too many minor actions were represented as nodes, the diagram became less useful as a high-level threat-modeling artifact. The final interpretation is that orphan-node handling should be treated as a validation and tuning problem rather than as a simple deletion rule. Nodes that represent major processes, data stores, trust-boundary crossings, or externally reachable components should be connected through explicit flows. In contrast, some controls may be better represented as evidence or attributes unless they clearly mediate data movement.

This behavior supports one of the design choices of the thesis: the LLM should not be responsible for creating or repairing the core DFD. The deterministic mapper gives the pipeline a stable architecture model, while validation checks make weaknesses in that model visible before threat identification and mitigation generation occur.

4.3 Candidate Risk Mapping and DREAD Scoring

The deterministic risk catalog mapped questionnaire-derived evidence to candidate risks across OWASP LLM, OWASP Web, and OWASP API categories. This stage provided a bounded risk universe before the local LLM was invoked. As a result, the LLM was not asked to freely invent the entire threat model. Instead, it operated over candidate risks that were already connected to questionnaire evidence.

This design improved traceability. Each candidate risk could be linked back to one or more questionnaire signals, such as public API exposure, retrieval from untrusted sources, tool execution capability, sensitive data handling, weak output controls, or missing monitoring. This made the later threat descriptions easier to audit because the model's generated text could be checked against deterministic evidence.

The DREAD scoring layer also behaved consistently because severity was computed from questionnaire-derived signals rather than generated by the LLM. This separation was important in cases where the model could produce persuasive but overly broad threat descriptions. The model could enrich a finding with an abuse path or mitigation explanation, but it could not increase or decrease final severity by changing the DREAD score. Therefore, the scoring layer preserved reproducibility even when the LLM output varied in wording.

The main limitation of this stage is that deterministic mapping depends on the completeness of the questionnaire and the quality of the risk catalog. If a relevant architectural feature is not captured by the questionnaire, the risk catalog cannot reliably map it to a candidate risk. Similarly, if a new attack pattern is not represented in the catalog, it may not appear as a primary scored finding. This limitation supports the sustainability objective of the thesis: new threat knowledge should be incorporated into the questionnaire, candidate-risk catalog, or fine-tuning data over time.

4.4 Behavior of the LLM-Supported Stages

The local LLM was most useful in the language-rich parts of the workflow. In threat identification, it could turn a deterministic candidate risk into a more system-specific description by referring to affected components, questionnaire-derived evidence, plausible abuse paths, and control gaps. In mitigation generation, it could produce more concrete engineering actions than a simple static template.

At the same time, the implementation confirmed that the LLM should remain inside a validation boundary. Free-form model output can introduce unsupported component references, generic mitigations, or risk descriptions that are not clearly grounded in the generated DFD. The grounding validator was therefore necessary for removing invalid risk codes, stripping references to non-existent DFD elements, downgrading unsupported findings, and preserving the distinction between deterministic scoring and model-generated explanation.

The overall result is that the local LLM layer worked best as an enrichment layer rather than as an authoritative decision layer. The deterministic stages created the architecture, selected candidate risks, and computed DREAD severity. The LLM then improved readability, specificity, and mitigation detail within those boundaries.

4.5 Fine-Tuning Setup

Fine-tuning experiments were conducted for two LLM-supported tasks: threat identification and mitigation generation. For each task, three dataset sizes were used: 500, 1000, and 1500 examples. The goal was to observe whether the local model could learn the structured output patterns required by the pipeline.

All runs used LoRA with rank 8 and alpha 16, a learning rate of 2×10^{-5}, dropout of 0.05, weight decay of 0.01, and early stopping with patience 2. The experiments were run for up to two epochs. Threat-identification runs used a per-device training batch size of 1 with gradient accumulation steps of 8, while mitigation-generation runs used a per-device training batch size of 2 with gradient accumulation steps of 4.

The reported accuracy values should be interpreted as automatic validation-generation metrics rather than expert-judged semantic correctness. Therefore, these results show optimization and task-format learning behavior, but they do not by themselves prove that the fine-tuned model produces superior threat models in an expert evaluation setting.

4.6 Threat Identification Fine-Tuning Results

Figure 4.1 shows that the threat-identification runs improved as the training set size increased. The 500-example run plateaued around 0.76 validation accuracy, while the 1000-example run reached approximately 0.89. The 1500-example run showed the strongest behavior, increasing steadily and plateauing around 0.96–0.97. This suggests that the threat-identification task benefited from additional task-specific examples, especially for learning the expected structured output format and recurring security patterns.

The validation loss in Figure 4.2 follows the same trend. The 500-example run decreased only modestly and then flattened, while the 1000-example and 1500-example runs continued to improve. The 1500-example run achieved the lowest validation loss, indicating more effective optimization under the same LoRA configuration.

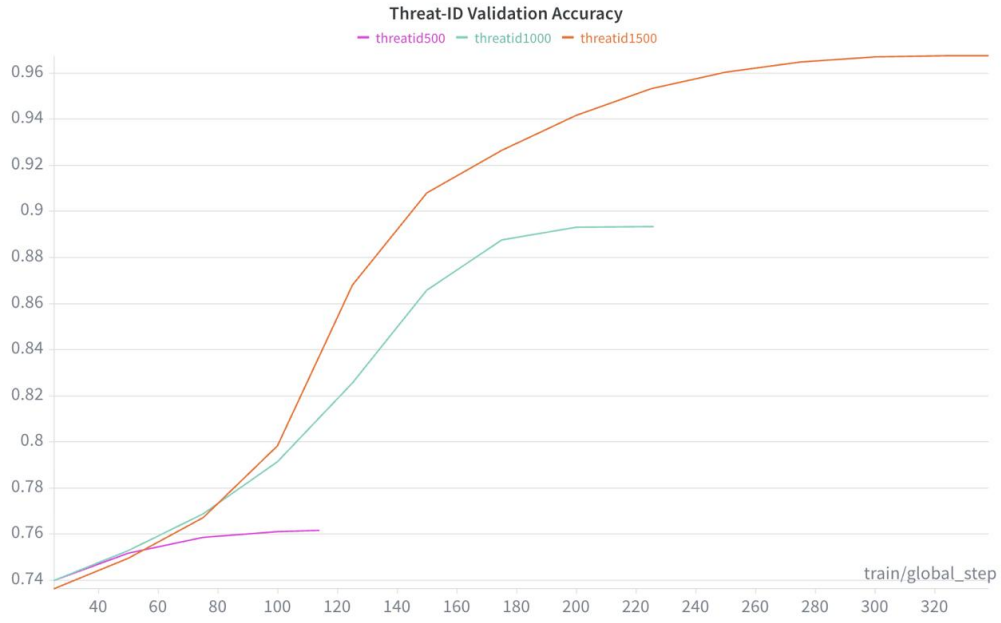

Figure 4.1 Threat identification validation accuracy across 500, 1000, and 1500 training examples

Figure 4.2 Threat identification validation loss across 500, 1000, and 1500 training examples

However, because the validation set is still based on prepared task data, this result should be interpreted as evidence of better task adaptation rather than as final evidence of real-world threat-identification quality.

4.7 Mitigation Generation Fine-Tuning Results

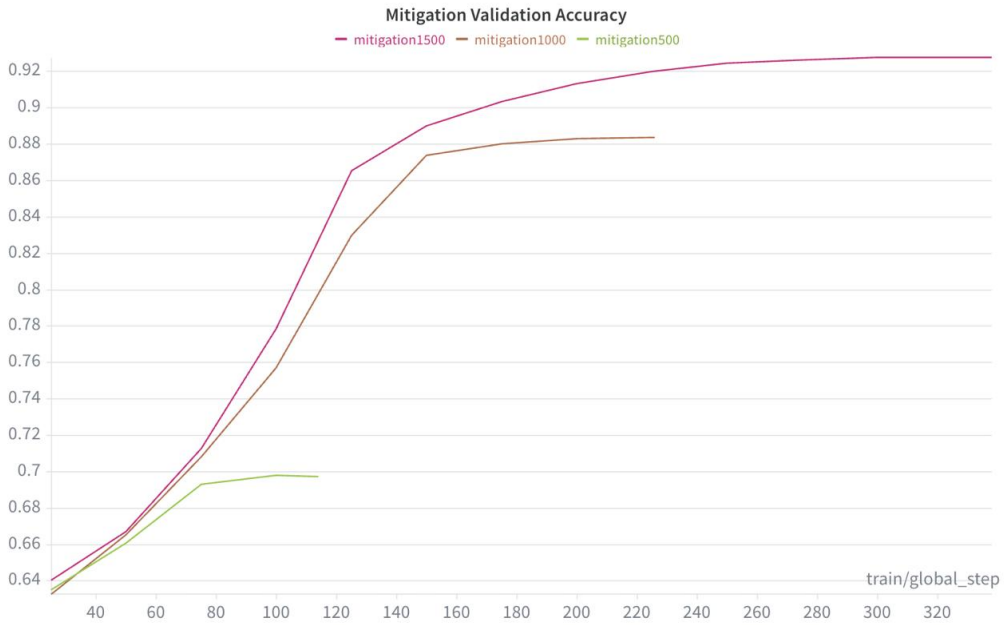

Figure 4.3 Mitigation generation validation accuracy across 500, 1000, and 1500 training examples

Figure 4.3 shows a similar but slightly lower trend for mitigation generation. The 500-example mitigation run plateaued below 0.70, while the 1000-example run reached approximately 0.88. The 1500-example run achieved the highest validation accuracy, reaching approximately 0.92–0.93. This indicates that mitigation generation also benefited from larger task-specific training data, although the task appears more difficult than threat identification because mitigation outputs require concrete actions, target components, validation steps, and links to evidence or control gaps.

The mitigation validation loss in Figure 4.4 supports the same interpretation. The 500-example run flattened at a relatively high loss, while the 1000-example and 1500-example runs continued to decrease. The 1500-example run produced the lowest validation loss, suggesting that additional examples helped the model learn the mitigation-generation format more effectively.

Compared with threat identification, mitigation generation showed lower final validation accuracy and higher final validation loss. This is consistent with the structure of the task. Threat identification is constrained by the deterministic candidate-risk set, whereas mitigation generation requires more action-oriented text and stronger alignment between risk code, affected component, control gap, validation step, and

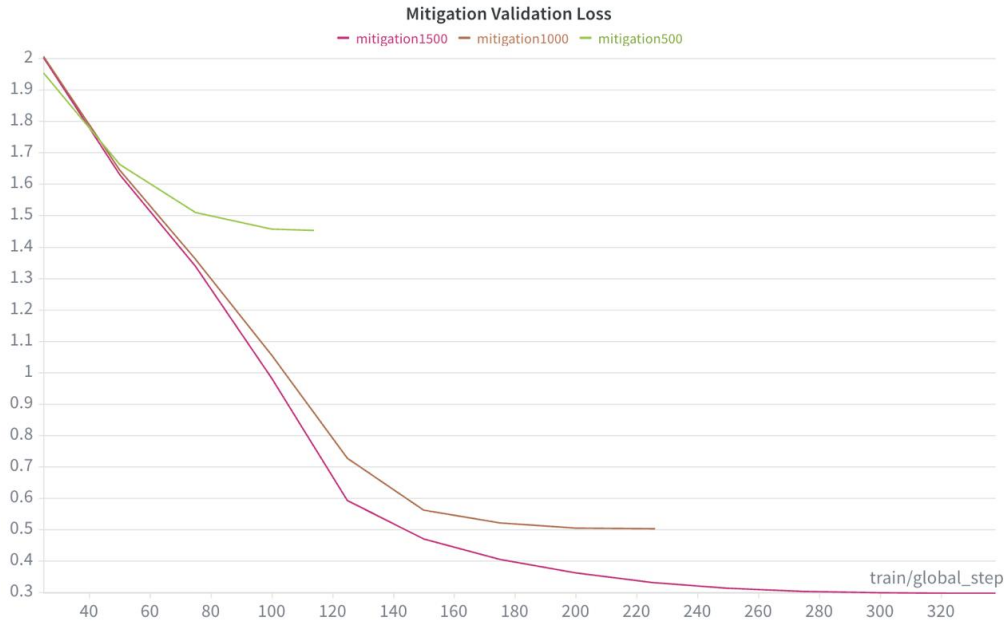

Figure 4.4 Mitigation generation validation loss across 500, 1000, and 1500 training examples

evidence.

4.8 Summary of Results

Overall, the results show that the proposed workflow is structurally feasible. The questionnaire-driven mapper can produce repeatable DFD artifacts, the deterministic risk catalog can connect questionnaire evidence to candidate risks, and the DREAD layer can preserve reproducible severity scoring without allowing the LLM to control prioritization.

The implementation also showed practical limitations. Static DFD generation requires careful tuning because overly strict rules may omit security-relevant elements, while overly permissive rules may turn too many actions into separate nodes. Orphan-node handling is therefore not only a cleanup step, but also an important design consideration for preserving both diagram readability and security relevance.

The fine-tuning diagnostics suggest that task-specific local adaptation is feasible. Increasing the training set size from 500 to 1500 examples improved automatic validation metrics for both threat identification and mitigation generation. Threat

identification showed stronger final validation behavior than mitigation generation, which is expected because mitigation generation is a more open-ended task. These results support the use of fine-tuning as an experimental extension, but they should be interpreted as training diagnostics rather than conclusive evidence of expert-level improvement.

The main finding from the results is therefore not that the LLM replaces deterministic threat modeling. Rather, the evidence supports the thesis design choice: deterministic stages should define the system model, candidate risks, validation rules, and DREAD severity, while the local LLM should be used as a bounded enrichment layer for system-specific threat descriptions and mitigation guidance.

5. DISCUSSION AND CONCLUSION

This thesis investigated to what extent local LLMs can improve the threat modeling process for LLM-integrated systems. The motivation for the work came from the observation that LLM-enabled applications are not only model-level artifacts, but system-level software architectures that combine web interfaces, APIs, retrieval components, model orchestration, external tools, data stores, and logging or monitoring pipelines. Prior work similarly argues that LLM-based systems should be assessed at the system level because conventional software risks, adversarial machine learning risks, and prompt-based attacks can interact across the full application architecture Nagaraja and Bahsi, 2025, 2026; Tete, 2024.

The proposed workflow addressed this problem by combining deterministic threat-modeling stages with bounded local LLM assistance. The deterministic stages derive a DFD from questionnaire answers, map questionnaire-derived evidence to candidate risks, validate grounding, and compute DREAD severity. The local LLM is used only for language-rich enrichment: producing system-specific threat descriptions, abuse paths, control-gap explanations, and mitigation guidance. This design follows the broader direction of LLM-assisted threat modeling research, while avoiding full reliance on unconstrained model generation for architecture construction or severity assignment Mollaeefar et al., 2024; Pathe, 2025; Wu et al., 2024.

5.1 Discussion

The main result of the thesis is that local LLMs are most useful when they operate inside a deterministic and validated threat-modeling pipeline. The LLM can improve the readability, specificity, and actionability of threat-modeling outputs, but

it should not be treated as the authoritative source for the system architecture, candidate risk universe, or final severity. This distinction is important because threat modeling depends on traceability: a threat should be connected to a component, data flow, trust boundary, evidence signal, or control gap. Without such grounding, generated findings may become generic or unsupported.

The static DFD mapper showed the value of keeping architecture generation deterministic. Traditional threat modeling often depends on manual DFD construction, which can be time-consuming and difficult for users without security expertise Hajrić et al., 2020; Shi et al., 2022; "Threat Modeling", 2018. In this thesis, the questionnaire-driven mapper reduced that burden by automatically creating a repeatable system model. However, the implementation also showed that static mapping requires careful tuning. If the mapping rules are too strict, the generated DFD may omit useful control-related elements. If they are too permissive, too many minor actions can become separate nodes, reducing readability. This shows that deterministic automation is valuable, but it still requires design judgment.

The candidate-risk mapping stage also supported the thesis objective. By mapping questionnaire evidence to OWASP Web, OWASP API, and OWASP LLM risk categories, the system created a unified risk view for hybrid LLM-enabled applications. This is important because LLM application risks often overlap with conventional web and API risks. For example, prompt injection, insecure output handling, excessive agency, and retrieval poisoning may lead to consequences similar to traditional injection, unauthorized access, or unsafe downstream actions "Formalizing and Benchmarking Prompt Injection Attacks and Defenses", 2023; "PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation", 2024; Tete, 2024.

The use of DREAD as a deterministic scoring layer was another important design decision. Prior work has applied DREAD to LLM-related risks, showing that it can be used to reason about impact, exploitability, reproducibility, affected users, and discoverability in this domain Tete, 2024; Zahid et al., 2024. This thesis extends that idea by grounding DREAD scoring in questionnaire-derived evidence rather than assigning severity through the LLM. As a result, severity remains reproducible even when the generated threat explanation changes.

The fine-tuning experiments suggest that task-specific adaptation of local models is feasible. The validation metrics improved as the training set size increased from 500 to 1500 examples for both threat identification and mitigation generation. This supports the idea that local models can learn the expected structure and terminology

of the pipeline. However, these results should be interpreted carefully. The reported metrics reflect automatic validation behavior and training diagnostics, not expert-judged semantic quality. Therefore, fine-tuning should be understood as a promising mechanism for improving structured output behavior rather than as conclusive proof that the model produces better threat models in all settings. Parameter-efficient methods such as LoRA make this kind of adaptation practical without retraining the entire model Dettmers et al., 2023; Hu et al., 2022.

5.2 Limitations

This thesis has several limitations. First, the evaluation is structural and automatic rather than expert-driven. The system was evaluated in terms of DFD integrity, candidate-risk coverage, grounding validity, schema conformance, system-specificity signals, mitigation actionability signals, and severity stability. These metrics are useful for testing whether the pipeline behaves consistently, but they do not replace expert review of threat quality.

Second, the questionnaire and risk catalog define the boundaries of the deterministic pipeline. If an important architectural feature is not captured by the questionnaire, or if a new attack pattern is not represented in the risk catalog, the system may fail to produce a corresponding primary candidate risk. This is a limitation of deterministic mapping, but it is also what makes the pipeline auditable.

Third, the local LLM remains dependent on the quality of its prompts, training data, and validation constraints. Even after fine-tuning, the model may produce unsupported claims, generic mitigations, or incomplete explanations. For this reason, the thesis treats the LLM as an enrichment layer rather than as an autonomous threat modeler.

Finally, the sustainability pipeline was considered as a design objective rather than evaluated as a long-term deployed monitoring system. The thesis describes how trusted security knowledge could be collected, normalized, and converted into task-specific training examples, but it does not evaluate continuous operation over an extended period.

5.3 Future Work

Future work should evaluate the generated threat models with expert reviewers or practitioner studies. Expert assessment would make it possible to measure semantic quality, usefulness, missing risks, false positives, and mitigation relevance more directly. This would complement the structural metrics used in this thesis.

Another direction is to expand the questionnaire and candidate-risk catalog. As LLM-enabled systems evolve, new architectural patterns and attack techniques will need to be represented. This is especially important for agentic systems, tool-use workflows, retrieval pipelines, memory systems, and multi-model orchestration.

Future work should also compare different local models and fine-tuning configurations under the same deterministic pipeline. Since the pipeline is model-agnostic, different base models, LoRA settings, dataset sizes, and prompt designs can be tested without changing the architecture generation or DREAD scoring logic.

Finally, the sustainability pipeline should be implemented and evaluated over time. A practical version of this component could monitor trusted sources, extract emerging LLM attack patterns, normalize them into structured examples, and periodically update the local model or risk catalog. This would help the system remain current as prompt injection, retrieval poisoning, tool misuse, and other LLM-specific risks continue to evolve.

5.4 Conclusion

This thesis presented a questionnaire-driven threat modeling workflow for generic LLM-enabled applications. The workflow combines deterministic DFD generation, candidate-risk mapping, grounding validation, and DREAD scoring with local LLM-supported threat identification and mitigation generation.

The central conclusion is that local LLMs can improve threat modeling when their role is carefully bounded. They are useful for producing system-specific explanations, abuse paths, control-gap descriptions, and mitigation guidance. However, they should not be responsible for defining the system architecture, selecting the

full risk universe, validating their own grounding, or assigning final severity. Those responsibilities are better handled by deterministic and auditable stages.

The thesis therefore supports a hybrid design: deterministic components provide reproducibility, traceability, and severity consistency, while the local LLM provides adaptable language-rich enrichment. Fine-tuning offers a practical path for improving the local model's task-specific behavior, and a sustainability-oriented update pipeline can help the system remain aligned with emerging LLM security risks. In this way, the proposed approach shows how local LLMs can contribute to threat modeling without replacing the structured reasoning and validation that security work requires.

BIBLIOGRAPHY

Bernsmed, K., Cruzes, D. S., Jaatun, M. G., & Iovan, M. (n.d.). Adopting threat modelling in agile software development projects [Preprint; metadata incomplete: year, DOI, URL, and page numbers unknown].

Bygdås, E., Jaatun, L. A., Antonsen, S. B., Ringen, A., & Eiring, E. (2021). Evaluating threat modeling tools: Microsoft tmt versus owasp threat dragon [Year and venue inferred from cross-references; exact venue, DOI, URL, and pages should be verified].

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, *36*, 10088–10115. https://arxiv.org/abs/2305.14314

Formalizing and benchmarking prompt injection attacks and defenses [Metadata incomplete: authors, venue, DOI, URL, and page numbers unknown]. (2023).

Hajrić, A., Smaka, T., Baraković, S., & Baraković Husić, J. (2020). Methods, methodologies, and tools for threat modeling with case study [Metadata incomplete: DOI, URL, and full page range unknown]. *Telfor Journal*.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). Lora: Low-rank adaptation of large language models. *International Conference on Learning Representations*. https://arxiv.org/abs/2106.09685

Mollaeefar, M., Bissoli, A., & Ranise, S. (2024). Pillar: An ai-powered privacy threat modeling tool [Exact publication details should be verified].

Nagaraja, N., & Bahsi, H. (2025). Cyber threat modeling of an llm-based healthcare system [Metadata incomplete: publisher, DOI, URL, and page numbers unknown]. *Proceedings of the 11th International Conference on Information Systems Security and Privacy*.

Nagaraja, N., & Bahsi, H. (2026). Where do llm-based systems break? a system-level security framework for risk assessment and treatment [Exact publication details should be verified].

Naik, N., Jenkins, P., Grace, P., Naik, D., Prajapat, S., & Song, J. (2024). A comparative analysis of threat modelling methods: Stride, dread, vast, pasta, octave, and linddun [Year inferred; venue, DOI, and URL unknown], 2–10.

Pathe, T. R. (2025). Towards threat modeling with large language models: Automating domain-specific language creation in meta attack language (mal) [Metadata incomplete: publication type, DOI, URL, and page numbers unknown].

Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation [Metadata incomplete: authors, venue, DOI, URL, and page numbers unknown]. (2024).

Securing large language models: Threats, vulnerabilities and responsible practices [Metadata incomplete: authors, exact venue, DOI, URL, and page numbers unknown]. (2024).

Shi, Z., Graffi, K., Starobinski, D., & Matyunin, N. (2022). Threat modeling tools: A taxonomy [Exact year and URL should be verified]. *IEEE Security & Privacy*, 1–12. https://doi.org/10.1109/MSEC.2021.3125229

Tete, S. B. (2024). Threat modelling and risk analysis for large language model (llm)-powered applications [Exact publication details should be verified].

Thompson, R. E., Red, M., Zhang, R., Kwon, Y., Dang, L., Pellegrini, C., Nesru, E., Jain, M., Chin, C., & Votipka, D. (2024). The threat modeling naturally tool: An interactive tool supporting more natural, flexible, and ad-hoc threat modeling [Metadata incomplete: paper URL, DOI, and page numbers unknown]. *USENIX Symposium on Usable Privacy and Security*. https://tsp.eecs.tufts.edu/tmnt

Threat modeling [Metadata incomplete: exact title, authors, venue, DOI, and page numbers were not visible]. (2018). https://ieeexplore.ieee.org/abstract/document/8587294

Unique security and privacy threats of large language models: A comprehensive survey [Metadata incomplete: authors, venue, DOI, URL, and page numbers unknown]. (2025).

Wu, T., Yang, S., Liu, S., Nguyen, D., Jang, S., & Abuadbba, A. (2024). *Threatmodeling-llm: Automating threat modeling using large language models for banking system* (Technical Report) (Metadata based on NotebookLM extraction; venue, DOI, URL, and page numbers should be verified). CSIRO Data61.

Zahid, F., Sewwandi, A., Brandon, L., Kumar, V., & Sinha, R. (2024). Securing educational llms: A generalised taxonomy of attacks on llms and dread risk assessment [Metadata incomplete: publication type, venue, DOI, URL, and page numbers unknown].

APPENDIX - METHODOLOGY ARTIFACTS

This appendix provides representative supplementary artifacts used to support the methodology described in Chapter 3. The material is included to make the questionnaire-driven pipeline, deterministic mappings, scoring logic, and LLM-supported stages more transparent without overloading the main text.

Questionnaire Structure

The system uses an adaptive questionnaire to collect information about the target LLM-enabled application. The questionnaire covers architecture, data handling, API exposure, retrieval behavior, tool use, model deployment, logging, monitoring, and DREAD-relevant risk signals.

Table A.1 shows representative questionnaire items.

Table A.1 Representative questionnaire items used by the threat modeling pipeline

Category	Example Question	Used For
Architecture	Does the application expose public API endpoints?	API risk mapping, DFD generation
Retrieval	Does the system retrieve external or user-controlled documents?	RAG and retrieval poisoning risks
Tool Use	Can the LLM invoke external tools or workflow actions?	Excessive agency and tool misuse risks
Data Handling	Does the system process sensitive user or business data?	Information disclosure and DREAD impact
Controls	Are input validation, output filtering, and monitoring implemented?	Control-gap detection

Example DFD Mapping Rules

The questionnaire answers are converted into a data-flow diagram through deterministic mapping rules. The LLM is not used to create the DFD.

```
If public_web_interface = true:
    create_external_entity("User")
    create_process("Web Frontend")
    create_data_flow("User", "Web Frontend", "User input")

If api_exposure = true:
    create_process("API Layer")
    create_data_flow("Web Frontend", "API Layer", "Application request")

If retrieval_enabled = true:
    create_process("Retrieval Component")
    create_data_store("Vector Store")
    create_data_flow("Retrieval Component", "Vector Store", "Context query")
```

Example Candidate Risk Mapping

Candidate risks are selected deterministically from questionnaire-derived evidence. The local LLM only enriches risks that already exist in the candidate set.

DREAD Scoring Signals

DREAD scoring is computed deterministically from questionnaire-derived signals. The LLM does not assign or modify final severity.

Table A.2 Representative candidate risk mapping rules

Risk Category	Triggering Evidence	Candidate Risk
OWASP LLM	User-controlled prompts are accepted	Prompt injection
OWASP LLM	Retrieval uses external or untrusted content	Retrieval poisoning
OWASP API	Public API endpoints are exposed	Broken object-level authorization
OWASP Web	User input is rendered in application output	Injection or unsafe output handling

Table A.3 Representative questionnaire-derived DREAD scoring signals

Dimension	Example Evidence Signals
Damage	Sensitive data processed, tool actions available, privileged functionality exposed
Reproducibility	Public endpoint, repeatable prompt path, persistent retrieval source
Exploitability	No authentication, weak input validation, untrusted content accepted
Affected Users	Multi-user deployment, shared data store, organization-wide workflow impact
Discoverability	Public interface, documented API, predictable prompt or retrieval behavior

Structured LLM Output Schema

The LLM-supported stages use structured outputs so that generated content can be validated against deterministic context.

```
{
  "risk_code": "LLM_PROMPT_INJECTION",
  "status": "applicable",
  "affected_elements": ["Web Frontend", "LLM Orchestrator"],
  "evidence": [
    "The system accepts user-controlled prompts",
    "The prompt is passed to the LLM orchestration layer"
  ],
  "abuse_path": "An attacker submits instructions that override intended behavior
```

```
  "control_gap": "No prompt filtering or instruction-boundary enforcement is pres
  "confidence": "high"
}
```


Example Mitigation Output

Mitigation generation is performed only after threats have been validated and scored.


```
{
  "risk_code": "LLM_PROMPT_INJECTION",
  "priority": "high",
  "target_component": "LLM Orchestrator",
  "mitigation": "Separate system instructions from user-controlled input and appl
  "validation_step": "Test adversarial prompts that attempt to override system in
  "linked_evidence": "User-controlled prompts are passed to the orchestration lay
}
```


Evaluation Metrics

The evaluation uses structural and grounding-oriented metrics rather than expert-judged semantic quality. The main metrics include DFD integrity, candidate-risk coverage, grounding validity, schema conformance, system-specificity, mitigation actionability, and severity stability.